

#Traffic Accident Prediction

Name : Rajni

Roll no : 20DCS018

Subject Code : CS-652

Subject : Machine Learning

Project Topic : Traffic Accident Prediction

Data Set : US Accidents (2016 - 2023)

(<https://www.kaggle.com/datasets/sobhanmoosavi/us-accidents/data>
(<https://www.kaggle.com/datasets/sobhanmoosavi/us-accidents/data>))

Data Set : Road Accident (United Kingdom (UK))

Dataset(<https://www.kaggle.com/datasets/devansodariya/road-accident-united-kingdom-uk-dataset>
(<https://www.kaggle.com/datasets/devansodariya/road-accident-united-kingdom-uk-dataset>))

Data Set : Hazardous-Driving-Spots-Around-the-

World(<https://www.openml.org/search?type=data&status=active&id=43372&sort=runs>
(<https://www.openml.org/search?type=data&status=active&id=43372&sort=runs>))

Data Set Analysis

```
In [2]: import pandas as pd  
import numpy as np
```

```
In [3]: file_path1 = 'RTC 2018_Leeds.csv'
file_path2 = 'Traffic accidents_2019_Leeds.csv'
file_path3 = 'only_road_accidents_data3.csv'
file_path4 = 'only_road_accidents_data_month2.csv'

# Read the CSV file into a DataFrame
data1 = pd.read_csv(file_path1)
data2 = pd.read_csv(file_path2)
data3 = pd.read_csv(file_path3)
data4 = pd.read_csv(file_path4)
```

```
In [8]: data1.columns
```

```
Out[8]: Index(['Accident Fields_Reference Number', 'Grid Ref: Easting',
              'Grid Ref: Northing', 'Number of Vehicles', 'Accident Date',
              'Time (24hr)', '1st Road Class', '1st Road Class & No', 'Road Surface',
              'Lighting Conditions', 'Weather Conditions', 'Local Authority',
              'Vehicle Fields_Reference Number', 'Vehicle Number', 'Type of Vehicle',
              'Casualty Fields_Reference Number', 'Casualty Veh No', 'Casualty Class',
              'Casualty Severity', 'Sex of Casualty', 'Age of Casualty'],
              dtype='object')
```

```
In [9]: data1.head()
```

```
Out[9]:
```

	Accident Fields_Reference Number	Grid Ref: Easting	Grid Ref: Northing	Number of Vehicles	Accident Date	Time (24hr)	1st Road Class	1st Road Class & No	Road Surface	Lig Cond
0	51B0230	433936	428874	1	11/01/2018	700	6	U	1	
1	51B0349	430076	433872	1	11/01/2018	855	6	U	2	
2	51B0349	430076	433872	1	11/01/2018	855	6	U	2	
3	51B0349	430076	433872	1	11/01/2018	855	6	U	2	
4	51B0349	430076	433872	1	11/01/2018	855	6	U	2	

5 rows × 21 columns

```
In [5]: data2.columns
```

```
Out[5]: Index(['Reference Number', 'Grid Ref: Easting', 'Grid Ref: Northing',
              'Number of Vehicles', 'Accident Date', 'Time (24hr)', '1st Road Class',
              '1st Road Class & No', 'Road Surface', 'Lighting Conditions',
              'Weather Conditions', 'Local Authority', 'Vehicle Number',
              'Type of Vehicle', 'Casualty Class', 'Casualty Severity',
              'Sex of Casualty', 'Age of Casualty'],
              dtype='object')
```

```
In [10]: data2.head()
```

```
Out[10]:
```

	Reference Number	Grid Ref: Easting	Grid Ref: Northing	Number of Vehicles	Accident Date	Time (24hr)	1st Road Class	1st Road Class & No	Road Surface	Lighting Conditions
0	58F1730	436147	434957	2	15/08/2019	1812	3	A6120	1	1
1	58F1730	436147	434957	2	15/08/2019	1812	3	A6120	1	1
2	58F1730	436147	434957	2	15/08/2019	1812	3	A6120	1	1
3	58F1730	436147	434957	2	15/08/2019	1812	3	A6120	1	1
4	58F1730	436147	434957	2	15/08/2019	1812	3	A6120	1	1

```
In [6]: data3.columns
```

```
Out[6]: Index(['STATE/UT', 'YEAR', '0-3 hrs. (Night)', '3-6 hrs. (Night)',  
              '6-9 hrs (Day)', '9-12 hrs (Day)', '12-15 hrs (Day)', '15-18 hrs (Da  
y)',  
              '18-21 hrs (Night)', '21-24 hrs (Night)', 'Total'],  
              dtype='object')
```

```
In [11]: data3.head()
```

```
Out[11]:
```

	STATE/UT	YEAR	0-3 hrs. (Night)	3-6 hrs. (Night)	6-9 hrs (Day)	9-12 hrs (Day)	12-15 hrs (Day)	15-18 hrs (Day)	18-21 hrs (Night)	21-24 hrs (Night)	Total
0	A & N Islands	2001	2	6	29	40	39	40	18	7	181
1	A & N Islands	2002	2	6	22	41	33	33	23	8	168
2	A & N Islands	2003	2	8	31	35	28	36	25	15	180
3	A & N Islands	2004	2	5	29	42	43	43	37	14	215
4	A & N Islands	2005	0	8	27	28	38	42	50	13	206

```
In [7]: data4.columns
```

```
Out[7]: Index(['STATE/UT', 'YEAR', 'JANUARY', 'FEBRUARY', 'MARCH', 'APRIL', 'MAY',  
              'JUNE', 'JULY', 'AUGUST', 'SEPTEMBER', 'OCTOBER', 'NOVEMBER',  
              'DECEMBER', 'TOTAL'],  
              dtype='object')
```

```
In [12]: data4.head()
```

Out[12]:

	STATE/UT	YEAR	JANUARY	FEBRUARY	MARCH	APRIL	MAY	JUNE	JULY	AUGUST	SEPTEMBER
0	A & N Islands	2001	8	23	15	15	14	19	14	19	
1	A & N Islands	2002	12	10	14	16	10	7	16	11	
2	A & N Islands	2003	19	13	15	13	13	12	8	16	
3	A & N Islands	2004	21	14	22	17	13	18	16	19	
4	A & N Islands	2005	19	21	22	17	13	19	21	14	



For DataSet1

```
In [28]: # Assuming 'data1' is your DataFrame
null_counts1 = data1.isnull().sum()

# Display the null value counts
# print(null_counts1)

df1 = pd.DataFrame(null_counts1)

# Check for missing data
missing_data1 = df1.isna()

# Count missing values in each column
missing_count1 = df1.isna().sum()

# Print the DataFrame with missing data
print("Original DataFrame:")
print(df1)
```

Original DataFrame:

	0
Accident Fields_Reference Number	0
Grid Ref: Easting	0
Grid Ref: Northing	0
Number of Vehicles	0
Accident Date	0
Time (24hr)	0
1st Road Class	0
1st Road Class & No	0
Road Surface	0
Lighting Conditions	0
Weather Conditions	0
Local Authority	0
Vehicle Fields_Reference Number	0
Vehicle Number	0
Type of Vehicle	0
Casualty Fields_Reference Number	0
Casualty Veh No	0
Casualty Class	0
Casualty Severity	0
Sex of Casualty	0
Age of Casualty	0

Missing Data

```
In [37]: # Print missing data status
print("\nMissing Data Status:")
print(missing_data1)
```

```
Missing Data Status:
0
Accident Fields_Reference Number False
Grid Ref: Easting                False
Grid Ref: Northing               False
Number of Vehicles               False
Accident Date                    False
Time (24hr)                      False
1st Road Class                   False
1st Road Class & No              False
Road Surface                     False
Lighting Conditions              False
Weather Conditions               False
Local Authority                  False
Vehicle Fields_Reference Number False
Vehicle Number                   False
Type of Vehicle                  False
Casualty Fields_Reference Number False
Casualty Veh No                  False
Casualty Class                   False
Casualty Severity                False
Sex of Casualty                  False
Age of Casualty                  False
```

Print missing data counts

```
In [38]: print("\nMissing Data Counts:")
print(missing_count1)
```

```
Missing Data Counts:
0    0
dtype: int64
```

Drop Rows with Missing Values

```
In [39]: df1_dropped1 = df1.dropna()
```

Fill Missing Values with a Specific Value

```
In [40]: df1_filled1 = df1.fillna(0)
```

Replace Missing Values with the Mean of the Column

```
In [41]: mean_fill1 = df1.fillna(df1.mean())
```

Print DataFrames after handling missing data

```
In [42]: print("\nDataFrame after dropping rows with missing data:")
print(df1_dropped1)
```

DataFrame after dropping rows with missing data:

	0
Accident Fields_Reference Number	0
Grid Ref: Easting	0
Grid Ref: Northing	0
Number of Vehicles	0
Accident Date	0
Time (24hr)	0
1st Road Class	0
1st Road Class & No	0
Road Surface	0
Lighting Conditions	0
Weather Conditions	0
Local Authority	0
Vehicle Fields_Reference Number	0
Vehicle Number	0
Type of Vehicle	0
Casualty Fields_Reference Number	0
Casualty Veh No	0
Casualty Class	0
Casualty Severity	0
Sex of Casualty	0
Age of Casualty	0

```
In [43]: # print("\nDataFrame after filling missing data with 0:")
print(df1_filled1)
```

	0
Accident Fields_Reference Number	0
Grid Ref: Easting	0
Grid Ref: Northing	0
Number of Vehicles	0
Accident Date	0
Time (24hr)	0
1st Road Class	0
1st Road Class & No	0
Road Surface	0
Lighting Conditions	0
Weather Conditions	0
Local Authority	0
Vehicle Fields_Reference Number	0
Vehicle Number	0
Type of Vehicle	0
Casualty Fields_Reference Number	0
Casualty Veh No	0
Casualty Class	0
Casualty Severity	0
Sex of Casualty	0
Age of Casualty	0

```
In [45]: # print("\nDataFrame after filling missing data with column mean:")
print(mean_fill1)
```

	0
Accident Fields_Reference Number	0
Grid Ref: Easting	0
Grid Ref: Northing	0
Number of Vehicles	0
Accident Date	0
Time (24hr)	0
1st Road Class	0
1st Road Class & No	0
Road Surface	0
Lighting Conditions	0
Weather Conditions	0
Local Authority	0
Vehicle Fields_Reference Number	0
Vehicle Number	0
Type of Vehicle	0
Casualty Fields_Reference Number	0
Casualty Veh No	0
Casualty Class	0
Casualty Severity	0
Sex of Casualty	0
Age of Casualty	0

```
In [46]: from sklearn.preprocessing import MinMaxScaler
```

```
In [47]: # Initialize the MinMaxScaler
scaler = MinMaxScaler()

# Fit and transform the DataFrame to normalize the data
normalized_data1 = scaler.fit_transform(df1)
```

```
In [48]: # Create a new DataFrame with the normalized values
normalized_df1 = pd.DataFrame(normalized_data1, columns=df1.columns)
```

Original DataFrame

```
In [49]: print("Original DataFrame:")
          print(df1)
```

Original DataFrame:

Accident Fields_Reference Number	0
Grid Ref: Easting	0
Grid Ref: Northing	0
Number of Vehicles	0
Accident Date	0
Time (24hr)	0
1st Road Class	0
1st Road Class & No	0
Road Surface	0
Lighting Conditions	0
Weather Conditions	0
Local Authority	0
Vehicle Fields_Reference Number	0
Vehicle Number	0
Type of Vehicle	0
Casualty Fields_Reference Number	0
Casualty Veh No	0
Casualty Class	0
Casualty Severity	0
Sex of Casualty	0
Age of Casualty	0

Normalized DataFrame

```
In [51]: # print("\nNormalized DataFrame:")  
print(normalized_df1)
```

```
      0  
0  0.0  
1  0.0  
2  0.0  
3  0.0  
4  0.0  
5  0.0  
6  0.0  
7  0.0  
8  0.0  
9  0.0  
10 0.0  
11 0.0  
12 0.0  
13 0.0  
14 0.0  
15 0.0  
16 0.0  
17 0.0  
18 0.0  
19 0.0  
20 0.0
```

Check for Duplicate Columns and Rows

```
In [11]: import pandas as pd

# Load your data from the CSV file
data1 = pd.read_csv('RTC 2018_Leeds.csv')

# Identify duplicate columns
duplicate_columns = []
seen_columns = set()
for column in data1.columns:
    if column in seen_columns:
        duplicate_columns.append(column)
    else:
        seen_columns.add(column)

# Identify duplicate rows
duplicate_rows = data1[data1.duplicated()]

if duplicate_columns:
    print("Duplicate columns found:")
    for column in duplicate_columns:
        print(column)
else:
    print("No duplicate columns found.")

if not duplicate_rows.empty:
    print("\nDuplicate rows found:")
    print(duplicate_rows)
else:
    print("No duplicate rows found.")
```

No duplicate columns found.

Duplicate rows found:

	Accident Fields_Reference Number	Grid Ref: Easting	Grid Ref: Northing
4	51B0349	430076	433872
230	5240945	432285	433647
335	5200005	419422	443129
336	5200005	419422	443129
390	53A0189	428111	426283
394	53A0824	430676	434874
582	54I1128	429654	431554
1039	56U0266	423867	437663
1043	56U0266	423867	437663
1886	5C90576	428937	429842
1993	5CV0814	432096	427519

	Number of Vehicles	Accident Date	Time (24hr)	1st Road Class	\
4	1	11/01/2018	855	6	
230	2	04/02/2018	435	3	
335	1	24/02/2018	5	6	
336	1	24/02/2018	5	6	
390	2	10/03/2018	215	3	
394	2	08/03/2018	2300	3	
582	1	06/04/2018	1535	6	
1039	2	30/06/2018	238	3	
1043	2	30/06/2018	238	3	
1886	2	09/12/2018	1047	6	
1993	2	31/12/2018	1345	1	

	1st Road Class & No	Road Surface	Lighting Conditions	...	\
4	U	2	1	...	
230	A64	1	4	...	
335	U	1	7	...	
336	U	1	7	...	
390	A650	2	4	...	
394	A61	1	4	...	
582	U	1	1	...	
1039	A6120	1	4	...	
1043	A6120	1	4	...	
1886	U	1	1	...	
1993	M1	2	1	...	

	Local Authority	Vehicle Fields_Reference Number	Vehicle Number	\
4	E08000035	51B0349	1	
230	E08000035	5240945	2	
335	E08000035	5200005	1	
336	E08000035	5200005	1	
390	E08000035	53A0189	1	
394	E08000035	53A0824	2	
582	E08000035	54I1128	1	
1039	E08000035	56U0266	1	
1043	E08000035	56U0266	1	
1886	E08000035	5C90576	2	
1993	E08000035	5CV0814	2	

Type of Vehicle	Casualty Fields_Reference Number	Casualty Veh No	\
-----------------	----------------------------------	-----------------	---

4	11	51B0349	1
230	8	5240945	2
335	9	5200005	1
336	9	5200005	1
390	8	53A0189	1
394	9	53A0824	2
582	11	54I1128	1
1039	9	56U0266	1
1043	9	56U0266	1
1886	9	5C90576	2
1993	9	5CV0814	2

	Casualty Class	Casualty Severity	Sex of Casualty	Age of Casualty
4	2	3	2	51
230	2	3	2	17
335	2	3	1	17
336	2	3	1	17
390	2	2	2	16
394	2	3	1	25
582	2	3	2	31
1039	2	2	2	17
1043	2	1	1	19
1886	2	3	1	52
1993	2	3	2	9

[11 rows x 21 columns]

Remove Duplicate Rows and Columns

```
In [12]: import pandas as pd

# Load your data from the CSV file
data1 = pd.read_csv('RTC 2018_Leeds.csv')

# Remove duplicate rows
data1 = data1.drop_duplicates()

# Remove duplicate columns
data1 = data1.loc[:, ~data1.columns.duplicated()]

# Save the cleaned data to a new CSV file
data1.to_csv('cleaned_data1.csv', index=False)
```

Check whether the dataset is imbalanced.

```
In [13]: import pandas as pd

file_path1 = 'RTC 2018_Leeds.csv'
data1 = pd.read_csv(file_path1)

df1 = pd.DataFrame(data1)

# Identify columns with imbalanced data (categorical columns with significantly
imbalanced_columns = []

for column in df1.columns:
    if df1[column].dtype == 'object':
        value_counts = df1[column].value_counts()
        if len(value_counts) <= 1 or value_counts.max() / value_counts.min() >= 10:
            imbalanced_columns.append(column)

print("Imbalanced Columns:")
print(imbalanced_columns)

Imbalanced Columns:
['Accident Date', '1st Road Class & No', 'Local Authority']
```

Generate Descriptive Analysis

```
In [49]: import matplotlib.pyplot as plt
import seaborn as sns

# Load your dataset into a DataFrame
file_path1 = 'RTC 2018_Leeds.csv'
data1 = pd.read_csv(file_path1)
```

Mean , Median or Mode

```
In [14]: import pandas as pd
import numpy as np
from statistics import mode

# Load your data from the CSV file
data1 = pd.read_csv('RTC 2018_Leeds.csv')

# Filter only the numeric columns
numeric_columns = data1.select_dtypes(include=[np.number])

# Calculate and display the mean, median, and mode for each numeric column
for column in numeric_columns.columns:
    mean = np.mean(numeric_columns[column])
    median = np.median(numeric_columns[column])

    # Flatten the column into a 1D array for mode calculation
    flat_data1 = numeric_columns[column].values.flatten()
    try:
        mode_value = mode(flat_data1)
    except StatisticsError:
        mode_value = "No unique mode found"

    print(f"Column: {column}")
    print(f"Mean: {mean}")
    print(f"Median: {median}")
    print(f"Mode: {mode_value}")
    print()
```

Column: Grid Ref: Easting
Mean: 429987.0320802005
Median: 429895.0
Mode: 431457

Column: Grid Ref: Northing
Mean: 434137.7739348371
Median: 433969.0
Mode: 430270

Column: Number of Vehicles
Mean: 1.919298245614035
Median: 2.0
Mode: 2

Column: Time (24hr)
Mean: 1409.3684210526317
Median: 1503.0
Mode: 1500

Column: 1st Road Class
Mean: 4.449624060150376
Median: 6.0
Mode: 6

Column: Road Surface
Mean: 1.2691729323308272
Median: 1.0
Mode: 1

Column: Lighting Conditions
Mean: 1.9393483709273183
Median: 1.0
Mode: 1

Column: Weather Conditions
Mean: 1.2275689223057644
Median: 1.0
Mode: 1

Column: Vehicle Number
Mean: 1.5042606516290726
Median: 1.0
Mode: 1

Column: Type of Vehicle
Mean: 8.167919799498748
Median: 9.0
Mode: 9

Column: Casualty Veh No
Mean: 1.5042606516290726
Median: 1.0
Mode: 1

Column: Casualty Class
Mean: 1.5543859649122806

Median: 1.0

Mode: 1

Column: Casualty Severity

Mean: 2.818045112781955

Median: 3.0

Mode: 3

Column: Sex of Casualty

Mean: 1.4105263157894736

Median: 1.0

Mode: 1

Column: Age of Casualty

Mean: 36.99398496240602

Median: 35.0

Mode: 23

Range, Variance, Standard Deviation or Interquartile Range (IQR)

```
In [15]: import pandas as pd
import numpy as np

# Load your data from the CSV file
data1 = pd.read_csv('RTC 2018_Leeds.csv')

# Select only numeric columns
numeric_data = data1.select_dtypes(include=[np.number])

# Calculate measures of dispersion for all numeric columns
dispersion_metrics = {
    'Column': [],
    'Range': [],
    'Variance': [],
    'Standard Deviation': [],
    'Interquartile Range (IQR)': []
}

for column in numeric_data.columns:
    dispersion_metrics['Column'].append(column)
    dispersion_metrics['Range'].append(np.ptp(numeric_data[column]))
    dispersion_metrics['Variance'].append(np.var(numeric_data[column]))
    dispersion_metrics['Standard Deviation'].append(np.std(numeric_data[column]))
    q1 = numeric_data[column].quantile(0.25)
    q3 = numeric_data[column].quantile(0.75)
    dispersion_metrics['Interquartile Range (IQR)'].append(q3 - q1)

# Create a summary DataFrame
dispersion_df = pd.DataFrame(dispersion_metrics)

# Print the results
print(dispersion_df)
```

	Column	Range	Variance	Standard Deviation \
0	Grid Ref: Easting	31181	2.718786e+07	5214.198280
1	Grid Ref: Northing	25577	2.132266e+07	4617.646964
2	Number of Vehicles	6	5.233118e-01	0.723403
3	Time (24hr)	2357	2.550746e+05	505.049129
4	1st Road Class	5	3.091573e+00	1.758287
5	Road Surface	4	2.909545e-01	0.539402
6	Lighting Conditions	6	2.537174e+00	1.592851
7	Weather Conditions	7	7.131247e-01	0.844467
8	Vehicle Number	6	3.752951e-01	0.612613
9	Type of Vehicle	96	3.303797e+01	5.747866
10	Casualty Veh No	6	3.752951e-01	0.612613
11	Casualty Class	2	5.628316e-01	0.750221
12	Casualty Severity	2	1.749125e-01	0.418225
13	Sex of Casualty	1	2.419945e-01	0.491929
14	Age of Casualty	94	3.553945e+02	18.851908

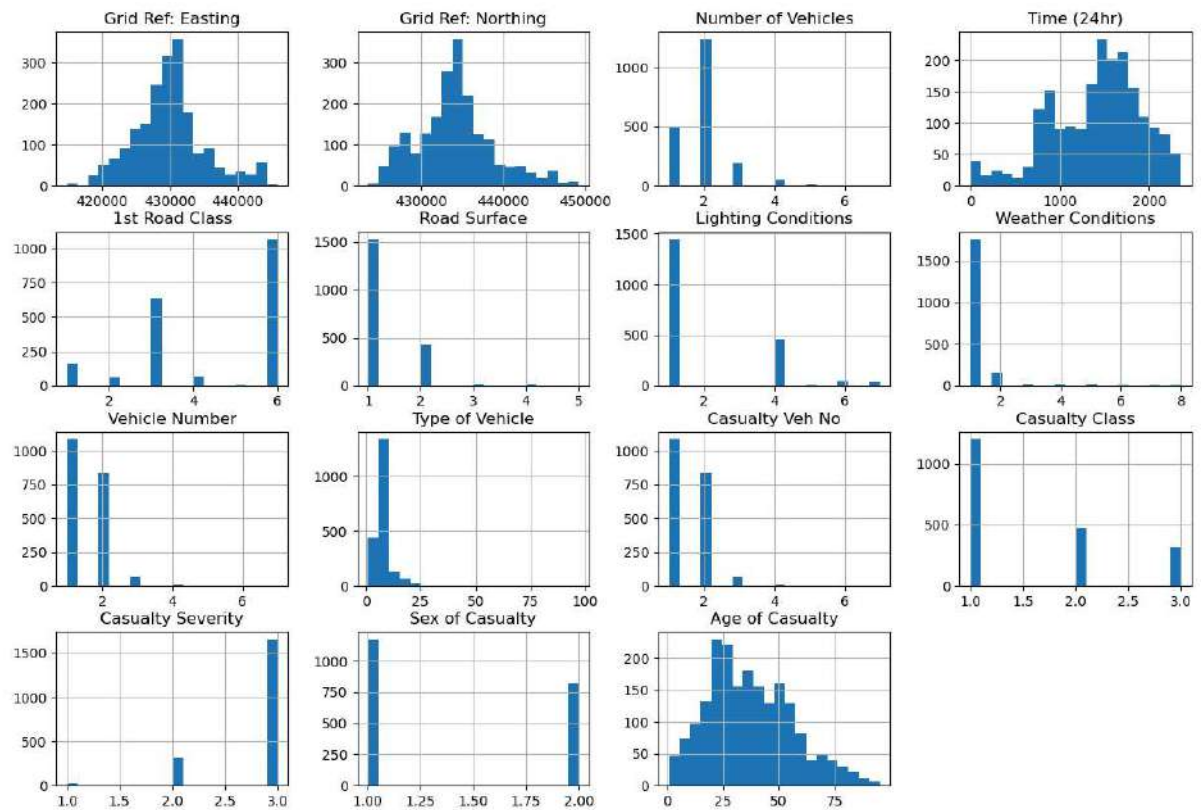
	Interquartile Range (IQR)
0	5262.5
1	4924.0
2	1.0
3	706.5
4	3.0
5	0.0
6	3.0
7	0.0
8	1.0
9	1.0
10	1.0
11	1.0
12	0.0
13	1.0
14	27.0

Frequency and Data Distributions using Histogram

```
In [25]: import matplotlib.pyplot as plt
import seaborn as sns

# Load your dataset into a DataFrame
file_path1 = 'RTC 2018_Leeds.csv'
data1 = pd.read_csv(file_path1)

# Histograms for all numeric columns
data1.hist(bins=20, figsize=(15, 10))
plt.show()
```



Summary statistics

```
In [97]: # Summary statistics
summary = data1.describe()
print(summary)
```

	Grid Ref: Easting	Grid Ref: Northing	Number of Vehicles	Time (24hr)
\				
count	1995.000000	1995.000000	1995.000000	1995.000000
mean	429987.032080	434137.773935	1.919298	1409.368421
std	5215.505588	4618.804704	0.723584	505.175755
min	414747.000000	423575.000000	1.000000	0.000000
25%	426900.000000	431337.000000	1.000000	1047.500000
50%	429895.000000	433969.000000	2.000000	1503.000000
75%	432162.500000	436261.000000	2.000000	1754.000000
max	445928.000000	449152.000000	7.000000	2357.000000

	1st Road Class	Road Surface	Lighting Conditions	Weather Conditions
\				
count	1995.000000	1995.000000	1995.000000	1995.000000
mean	4.449624	1.269173	1.939348	1.227569
std	1.758728	0.539537	1.593250	0.844679
min	1.000000	1.000000	1.000000	1.000000
25%	3.000000	1.000000	1.000000	1.000000
50%	6.000000	1.000000	1.000000	1.000000
75%	6.000000	1.000000	4.000000	1.000000
max	6.000000	5.000000	7.000000	8.000000

	Vehicle Number	Type of Vehicle	Casualty Veh No	Casualty Class	\
count	1995.000000	1995.000000	1995.000000	1995.000000	
mean	1.504261	8.167920	1.504261	1.554386	
std	0.612767	5.749308	0.612767	0.750409	
min	1.000000	1.000000	1.000000	1.000000	
25%	1.000000	8.000000	1.000000	1.000000	
50%	1.000000	9.000000	1.000000	1.000000	
75%	2.000000	9.000000	2.000000	2.000000	
max	7.000000	97.000000	7.000000	3.000000	

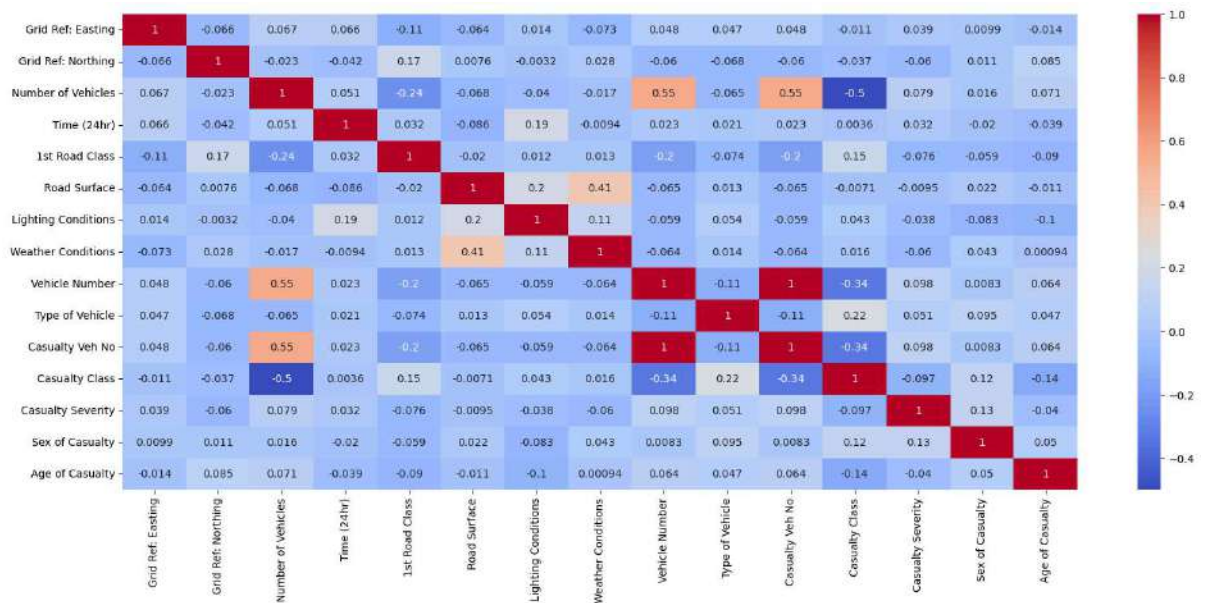
	Casualty Severity	Sex of Casualty	Age of Casualty
count	1995.000000	1995.000000	1995.000000
mean	2.818045	1.410526	36.993985
std	0.418330	0.492053	18.856635
min	1.000000	1.000000	1.000000
25%	3.000000	1.000000	23.000000
50%	3.000000	1.000000	35.000000
75%	3.000000	2.000000	50.000000
max	3.000000	2.000000	95.000000

Correlation heatmap for numeric columns

```
In [103]: # Correlation heatmap for numeric columns
correlation_matrix = data1.corr()
plt.figure(figsize=(20, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.show()
```

C:\Users\lenovo\AppData\Local\Temp\ipykernel_224\2338908580.py:2: FutureWarning: The default value of numeric_only in DataFrame.corr is deprecated. In a future version, it will default to False. Select only valid columns or specify the value of numeric_only to silence this warning.

```
correlation_matrix = data1.corr()
```

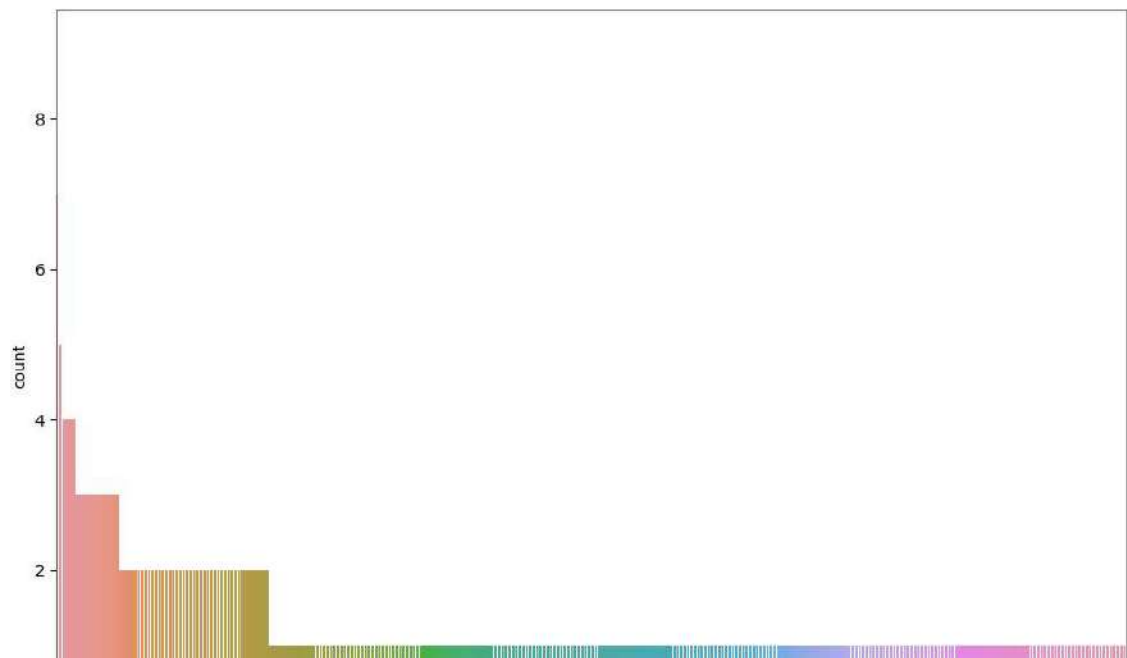


Frequency distribution for categorical columns

```
In [11]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

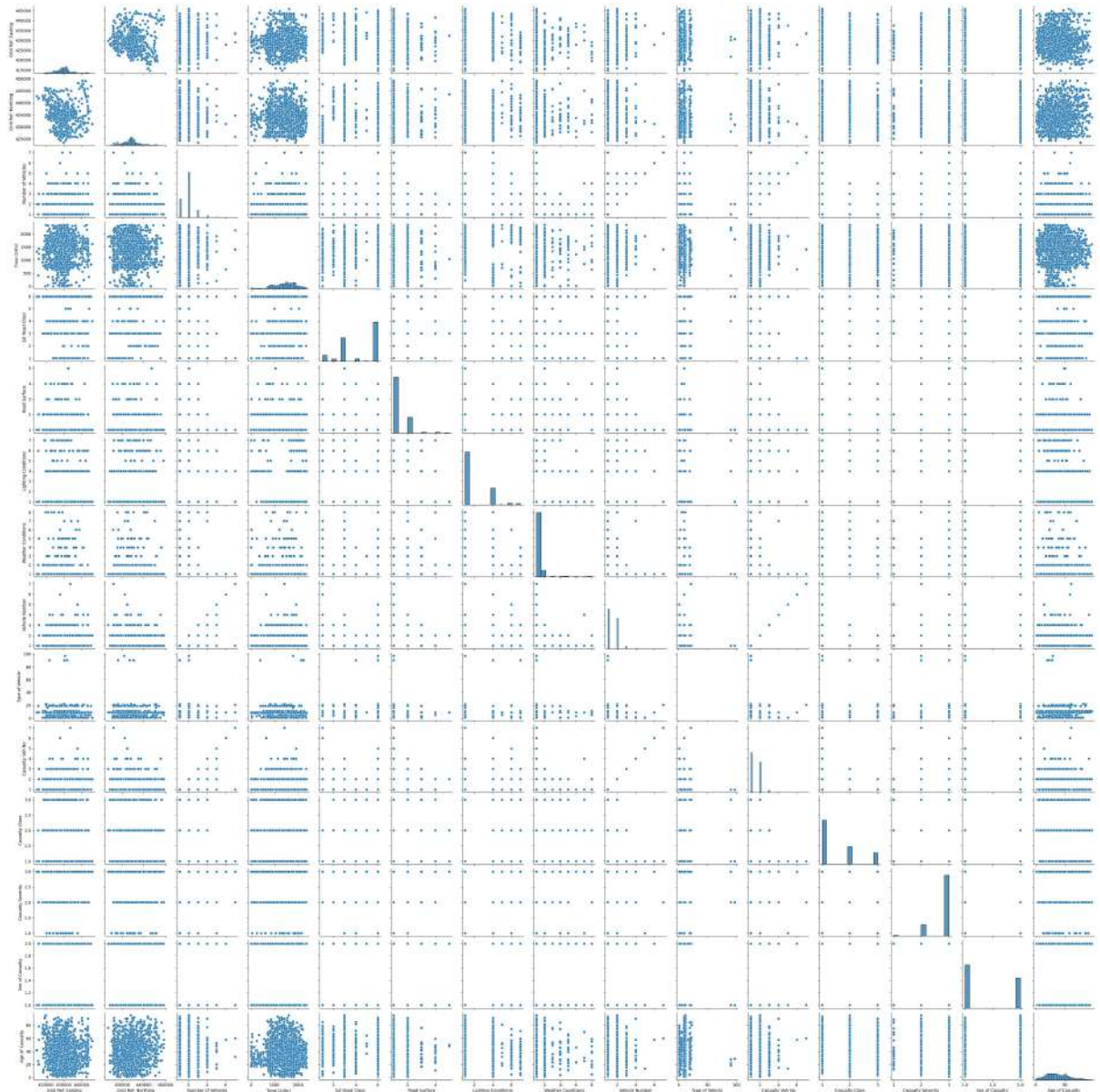
file_path1 = 'RTC 2018_Leeds.csv'
data1 = pd.read_csv(file_path1)

# Frequency distribution for categorical columns
categorical_columns = data1.select_dtypes(include='object')
for column in categorical_columns:
    plt.figure(figsize=(12, 8))
    sns.countplot(data=data1, x=column, order=data1[column].value_counts().index)
    plt.xticks(rotation=45)
    plt.show()
```



Pairwise scatter plot matrix for numeric columns

```
In [12]: # Pairwise scatter plot matrix for numeric columns
sns.pairplot(data1, kind='scatter')
plt.show()
```



Find missing values

```
In [18]: # Check for missing values
missing_values = data1.isnull().sum()
print("Missing Values:")
print(missing_values)
```

```
Missing Values:
Accident Fields_Reference Number    0
Grid Ref: Easting                   0
Grid Ref: Northing                  0
Number of Vehicles                   0
Accident Date                       0
Time (24hr)                         0
1st Road Class                      0
1st Road Class & No                 0
Road Surface                        0
Lighting Conditions                 0
Weather Conditions                  0
Local Authority                     0
Vehicle Fields_Reference Number     0
Vehicle Number                      0
Type of Vehicle                     0
Casualty Fields_Reference Number    0
Casualty Veh No                     0
Casualty Class                      0
Casualty Severity                   0
Sex of Casualty                     0
Age of Casualty                     0
dtype: int64
```

Check data types

```
In [16]: # Check data types
file_path1 = 'RTC 2018_Leeds.csv'
data1 = pd.read_csv(file_path1)

data1_types = data1.dtypes
print("\nData Types:")
print(data1_types)
```

```
Data Types:
Accident Fields_Reference Number    object
Grid Ref: Easting                    int64
Grid Ref: Northing                   int64
Number of Vehicles                   int64
Accident Date                        object
Time (24hr)                          int64
1st Road Class                       int64
1st Road Class & No                  object
Road Surface                         int64
Lighting Conditions                  int64
Weather Conditions                   int64
Local Authority                      object
Vehicle Fields_Reference Number      object
Vehicle Number                       int64
Type of Vehicle                      int64
Casualty Fields_Reference Number     object
Casualty Veh No                      int64
Casualty Class                       int64
Casualty Severity                    int64
Sex of Casualty                      int64
Age of Casualty                      int64
dtype: object
```

Check the number of unique values for each column

```
In [20]: unique_values = data1.nunique()  
print("\nUnique Values:")  
print(unique_values)
```

```
Unique Values:  
Accident Fields_Reference Number    1548  
Grid Ref: Easting                   1432  
Grid Ref: Northing                  1422  
Number of Vehicles                    7  
Accident Date                       354  
Time (24hr)                         621  
1st Road Class                       6  
1st Road Class & No                  50  
Road Surface                         5  
Lighting Conditions                 5  
Weather Conditions                  8  
Local Authority                     1  
Vehicle Fields_Reference Number    1548  
Vehicle Number                      7  
Type of Vehicle                     16  
Casualty Fields_Reference Number    1548  
Casualty Veh No                     7  
Casualty Class                      3  
Casualty Severity                   3  
Sex of Casualty                     2  
Age of Casualty                     94  
dtype: int64
```

Check for duplicate rows

```
In [21]: duplicate_rows = data1.duplicated().sum()  
print("\nDuplicate Rows:")  
print(duplicate_rows)
```

```
Duplicate Rows:  
11
```

Basic summary statistics

```
In [43]: summary_stats = data1.describe()
print("\nSummary Statistics:")
print(summary_stats)
```

Summary Statistics:

	Grid Ref: Easting	Grid Ref: Northing	Number of Vehicles	Time (24hr)
\				
count	1995.000000	1995.000000	1995.000000	1995.000000
mean	429987.032080	434137.773935	1.919298	1409.368421
std	5215.505588	4618.804704	0.723584	505.175755
min	414747.000000	423575.000000	1.000000	0.000000
25%	426900.000000	431337.000000	1.000000	1047.500000
50%	429895.000000	433969.000000	2.000000	1503.000000
75%	432162.500000	436261.000000	2.000000	1754.000000
max	445928.000000	449152.000000	7.000000	2357.000000

	1st Road Class	Road Surface	Lighting Conditions	Weather Conditions
\				
count	1995.000000	1995.000000	1995.000000	1995.000000
mean	4.449624	1.269173	1.939348	1.227569
std	1.758728	0.539537	1.593250	0.844679
min	1.000000	1.000000	1.000000	1.000000
25%	3.000000	1.000000	1.000000	1.000000
50%	6.000000	1.000000	1.000000	1.000000
75%	6.000000	1.000000	4.000000	1.000000
max	6.000000	5.000000	7.000000	8.000000

	Vehicle Number	Type of Vehicle	Casualty Veh No	Casualty Class	\
count	1995.000000	1995.000000	1995.000000	1995.000000	
mean	1.504261	8.167920	1.504261	1.554386	
std	0.612767	5.749308	0.612767	0.750409	
min	1.000000	1.000000	1.000000	1.000000	
25%	1.000000	8.000000	1.000000	1.000000	
50%	1.000000	9.000000	1.000000	1.000000	
75%	2.000000	9.000000	2.000000	2.000000	
max	7.000000	97.000000	7.000000	3.000000	

	Casualty Severity	Sex of Casualty	Age of Casualty
count	1995.000000	1995.000000	1995.000000
mean	2.818045	1.410526	36.993985
std	0.418330	0.492053	18.856635
min	1.000000	1.000000	1.000000
25%	3.000000	1.000000	23.000000
50%	3.000000	1.000000	35.000000
75%	3.000000	2.000000	50.000000
max	3.000000	2.000000	95.000000

Check for outliers in file

```
In [30]: import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
data = pd.read_csv('RTC 2018_Leeds.csv')

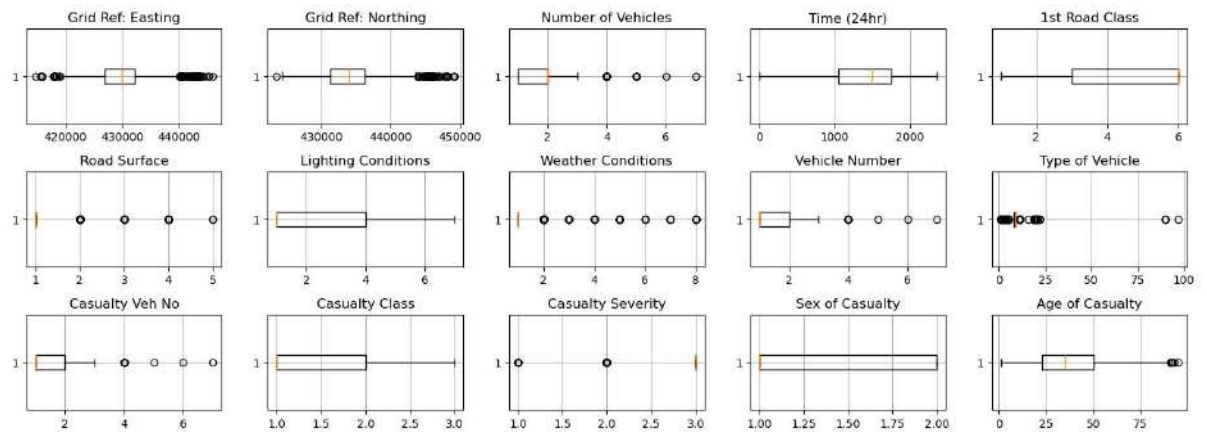
# Select columns with numeric data types (excluding non-numeric columns)
numeric_columns = data.select_dtypes(include=['int64', 'float64'])

# Set the figure size
plt.figure(figsize=(15, 7))

# Create box plots for each numeric column
for i, column in enumerate(numeric_columns.columns):
    plt.subplot(4, 5, i + 1)
    plt.boxplot(data[column], vert=False)
    plt.title(column)
    plt.grid(True)

# Adjust the layout
plt.tight_layout()

# Show the box plots
plt.show()
```



Remove Outliers

```
In [17]: import pandas as pd
import numpy as np

def remove_outliers(file_path, threshold=1.5):
    # Read the CSV file into a DataFrame
    data1 = pd.read_csv(file_path)

    # Create a copy of the original data
    original_data = data1.copy()

    # Detect outliers using the IQR method
    Q1 = data1.quantile(0.25, numeric_only=True)
    Q3 = data1.quantile(0.75, numeric_only=True)

    IQR = Q3 - Q1

    lower_bound = Q1 - threshold * IQR
    upper_bound = Q3 + threshold * IQR

    # Align DataFrames and create a boolean mask to identify outliers
    left, right = data1.align(lower_bound, axis=1, copy=False)
    outliers_mask = ((left < right) | (left > right))

    # Remove rows that contain outliers
    data_no_outliers = data1[~outliers_mask.any(axis=1)]

    return original_data, data_no_outliers

# Example usage
file_path = 'RTC 2018_Leeds.csv'
original_data, data_no_outliers = remove_outliers(file_path)

# Save the data without outliers to a new CSV file
data_no_outliers.to_csv('data_no_outliers.csv', index=False)
# print(data_no_outliers)
```

Data adequacy assessment summary

```
In [42]: if missing_values.sum() == 0 and duplicate_rows == 0:
        print("\nData appears to be adequate.")
    else:
        print("\nData has issues that need to be addressed.")
```

Data has issues that need to be addressed.

Linear Regression Models for DataSet 1

In []: Linear Regression: Simple linear regression can be used to predict the number of

```
In [1]: # Import necessary libraries
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Load your dataset (replace 'your_dataset.csv' with the actual dataset file)
data = pd.read_csv('RTC 2018_Leeds.csv')

# Select the features and target variable
X = data[['Number of Vehicles', '1st Road Class', 'Road Surface', 'Lighting Condition']]
y = data['Casualty Severity']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create a Linear Regression model
model = LinearRegression()

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# Print the model coefficients and evaluation metrics
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
print("Mean Squared Error:", mse)
print("R-squared:", r2)
```

```
Coefficients: [ 0.04428294 -0.00875663  0.02398015 -0.00555943 -0.04804786]
Intercept: 2.806133469000831
Mean Squared Error: 0.15595556756117437
R-squared: -0.007968590098511141
```

Decision Trees for prediction of accident

In []: Decision Trees: Decision trees can be used to identify the factors that contri

```
In [2]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load your dataset (replace 'your_dataset.csv' with the actual dataset file)
data = pd.read_csv('RTC 2018_Leeds.csv')

# Select the features and target variable
X = data[['Number of Vehicles', '1st Road Class', 'Road Surface', 'Lighting Con
y = data['Casualty Severity']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create a Decision Tree classifier
model = DecisionTreeClassifier(random_state=42)

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

# Print the model's accuracy and classification report
print("Accuracy:", accuracy)
print("Classification Report:\n", report)
```

Accuracy: 0.8320802005012531

Classification Report:

	precision	recall	f1-score	support
1	0.00	0.00	0.00	4
2	0.30	0.05	0.09	56
3	0.85	0.97	0.91	339
accuracy			0.83	399
macro avg	0.38	0.34	0.33	399
weighted avg	0.77	0.83	0.78	399

SVM Model

```
In [ ]: Effective in High-Dimensional Spaces:  
Robustness to Outliers  
Margin Maximization  
Well-Separated Data
```

```

In [3]: # Import necessary Libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# Load your dataset (replace 'your_dataset.csv' with the actual dataset file)
data = pd.read_csv('RTC 2018_Leeds.csv')

# Select the features and target variable
X = data[['Number of Vehicles', '1st Road Class', 'Road Surface', 'Lighting Con
y = data['Casualty Severity']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create an SVM classifier
model = SVC(kernel='linear', random_state=42) # You can choose the kernel type

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

# Print the model's accuracy and classification report
print("Accuracy:", accuracy)
print("Classification Report:\n", report)

```

Accuracy: 0.849624060150376

Classification Report:

	precision	recall	f1-score	support
1	0.00	0.00	0.00	4
2	0.00	0.00	0.00	56
3	0.85	1.00	0.92	339
accuracy			0.85	399
macro avg	0.28	0.33	0.31	399
weighted avg	0.72	0.85	0.78	399

```
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and bei
ng set to 0.0 in labels with no predicted samples. Use `zero_division` parame
ter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and bei
ng set to 0.0 in labels with no predicted samples. Use `zero_division` parame
ter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and bei
ng set to 0.0 in labels with no predicted samples. Use `zero_division` parame
ter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
```

Random Forest for Dataset 1

```
In [4]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load your dataset (replace 'your_dataset.csv' with the actual dataset file)
data = pd.read_csv('RTC 2018_Leeds.csv')

# Select the features and target variable
X = data[['Number of Vehicles', '1st Road Class', 'Road Surface', 'Lighting Con
y = data['Casualty Severity']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create a Random Forest classifier
model = RandomForestClassifier(n_estimators=100, random_state=42) # You can ad

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

# Print the model's accuracy and classification report
print("Accuracy:", accuracy)
print("Classification Report:\n", report)
```

Accuracy: 0.8395989974937343

Classification Report:

	precision	recall	f1-score	support
1	0.00	0.00	0.00	4
2	0.43	0.05	0.10	56
3	0.85	0.98	0.91	339
accuracy			0.84	399
macro avg	0.43	0.34	0.34	399
weighted avg	0.79	0.84	0.79	399

Evaluation Metrics for DataSet 1

Accuracy Metrics

```
In [7]: # Import necessary Libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load your dataset
data = pd.read_csv('RTC 2018_Leeds.csv')

# Select the features and target variable
X = data[['Number of Vehicles', '1st Road Class', 'Road Surface', 'Lighting Con
y = data['Casualty Severity']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create a Decision Tree classifier
model = DecisionTreeClassifier(random_state=42)

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

# Print the accuracy
print("Accuracy:", accuracy)
```

Accuracy: 0.8320802005012531

Precision and Recall Metrics

```
In [8]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import precision_score, recall_score

# Load your dataset
data = pd.read_csv('RTC 2018_Leeds.csv')

# Select the features and target variable
X = data[['Number of Vehicles', '1st Road Class', 'Road Surface', 'Lighting Cor
y = data['Casualty Severity']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create a Decision Tree classifier
model = DecisionTreeClassifier(random_state=42)

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Calculate Precision and Recall
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')

# Print the Precision and Recall
print("Precision:", precision)
print("Recall:", recall)
```

Precision: 0.7681476418318524

Recall: 0.8320802005012531

Mean Absolute Error (MAE) or Mean Squared Error (MSE) Metrics

```
In [9]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error

# Load your dataset
data = pd.read_csv('RTC 2018_Leeds.csv')

# Select the features and target variable
X = data[['Number of Vehicles', '1st Road Class', 'Road Surface', 'Lighting Con
y = data['Casualty Severity']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create a Decision Tree Regressor (for regression)
model = DecisionTreeRegressor(random_state=42)

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Calculate Mean Absolute Error (MAE)
mae = mean_absolute_error(y_test, y_pred)

# Calculate Mean Squared Error (MSE)
mse = mean_squared_error(y_test, y_pred)

# Print MAE and MSE
print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
```

Mean Absolute Error (MAE): 0.27514764336752046

Mean Squared Error (MSE): 0.1836690358711977

Log Loss (Logarithmic Loss) Metrics

```
In [10]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import log_loss

# Load your dataset
data = pd.read_csv('RTC 2018_Leeds.csv')

# Select the features and target variable
X = data[['Number of Vehicles', '1st Road Class', 'Road Surface', 'Lighting Con
y = data['Casualty Severity']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create a Random Forest classifier
model = RandomForestClassifier(n_estimators=100, random_state=42)

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data (probabilities)
y_prob = model.predict_proba(X_test)

# Calculate Log Loss
logloss = log_loss(y_test, y_prob)

# Print Log Loss
print("Log Loss:", logloss)
```

Log Loss: 0.8781733164393722

Root Mean Squared Error (RMSE) Metrics

```
In [11]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error
import numpy as np

# Load your dataset
data = pd.read_csv('RTC 2018_Leeds.csv')

# Select the features and target variable
X = data[['Number of Vehicles', '1st Road Class', 'Road Surface', 'Lighting Con
y = data['Casualty Severity']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create a Decision Tree Regressor (for regression)
model = DecisionTreeRegressor(random_state=42)

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Calculate Mean Squared Error (MSE)
mse = mean_squared_error(y_test, y_pred)

# Calculate Root Mean Squared Error (RMSE)
rmse = np.sqrt(mse)

# Print RMSE
print("Root Mean Squared Error (RMSE):", rmse)
```

Root Mean Squared Error (RMSE): 0.42856625610423144

For DataSet2

```
In [19]: file_path2 = 'Traffic accidents_2019_Leeds.csv'
data2 = pd.read_csv(file_path2)
```

```

In [20]: # Assuming 'data2' is your DataFrame
null_counts2 = data2.isnull().sum()

# Display the null value counts
# print(null_counts2)

df2 = pd.DataFrame(null_counts2)

# Check for missing data
missing_data2 = df2.isna()

# Count missing values in each column
missing_count2 = df2.isna().sum()

# Print the DataFrame with missing data
print("Original DataFrame:")
print(df2)

```

Original DataFrame:

	0
Reference Number	0
Grid Ref: Easting	0
Grid Ref: Northing	0
Number of Vehicles	0
Accident Date	0
Time (24hr)	0
1st Road Class	0
1st Road Class & No	0
Road Surface	0
Lighting Conditions	0
Weather Conditions	0
Local Authority	0
Vehicle Number	0
Type of Vehicle	0
Casualty Class	0
Casualty Severity	0
Sex of Casualty	0
Age of Casualty	0

Missing Data

```
In [22]: # Print missing data status
print("\nMissing Data Status:")
print(missing_data2)
```

```
Missing Data Status:
0
Reference Number      False
Grid Ref: Easting     False
Grid Ref: Northing    False
Number of Vehicles    False
Accident Date         False
Time (24hr)           False
1st Road Class        False
1st Road Class & No   False
Road Surface          False
Lighting Conditions   False
Weather Conditions    False
Local Authority        False
Vehicle Number         False
Type of Vehicle        False
Casualty Class         False
Casualty Severity      False
Sex of Casualty        False
Age of Casualty        False
```

Print missing data counts

```
In [24]: print("\nMissing Data Counts:")
print(missing_count2)
```

```
Missing Data Counts:
0    0
dtype: int64
```

Drop Rows with Missing Values

```
In [36]: df2_dropped2 = df2.dropna()
```

Fill Missing Values with a Specific Value

```
In [37]: df2_filled2 = df2.fillna(0)
```

Replace Missing Values with the Mean of the Column

```
In [38]: mean_fill2 = df2.fillna(df2.mean())
```

Print DataFrames after handling missing data

```
In [39]: print("\nDataFrame after dropping rows with missing data:")  
print(df2_dropped2)
```

DataFrame after dropping rows with missing data:

	0
Reference Number	0
Grid Ref: Easting	0
Grid Ref: Northing	0
Number of Vehicles	0
Accident Date	0
Time (24hr)	0
1st Road Class	0
1st Road Class & No	0
Road Surface	0
Lighting Conditions	0
Weather Conditions	0
Local Authority	0
Vehicle Number	0
Type of Vehicle	0
Casualty Class	0
Casualty Severity	0
Sex of Casualty	0
Age of Casualty	0

```
In [40]: # print("\nDataFrame after filling missing data with 0:")
print(df2_filled2)
```

	0
Reference Number	0
Grid Ref: Easting	0
Grid Ref: Northing	0
Number of Vehicles	0
Accident Date	0
Time (24hr)	0
1st Road Class	0
1st Road Class & No	0
Road Surface	0
Lighting Conditions	0
Weather Conditions	0
Local Authority	0
Vehicle Number	0
Type of Vehicle	0
Casualty Class	0
Casualty Severity	0
Sex of Casualty	0
Age of Casualty	0

```
In [41]: # print("\nDataFrame after filling missing data with column mean:")
print(mean_fill2)
```

	0
Reference Number	0
Grid Ref: Easting	0
Grid Ref: Northing	0
Number of Vehicles	0
Accident Date	0
Time (24hr)	0
1st Road Class	0
1st Road Class & No	0
Road Surface	0
Lighting Conditions	0
Weather Conditions	0
Local Authority	0
Vehicle Number	0
Type of Vehicle	0
Casualty Class	0
Casualty Severity	0
Sex of Casualty	0
Age of Casualty	0

```
In [42]: from sklearn.preprocessing import MinMaxScaler
# Initialize the MinMaxScaler
scaler = MinMaxScaler()

# Fit and transform the DataFrame to normalize the data
normalized_data2 = scaler.fit_transform(df2)
# Create a new DataFrame with the normalized values
normalized_df2 = pd.DataFrame(normalized_data2, columns=df2.columns)
```

Original DataFrame

```
In [43]: print("Original DataFrame:")
print(df2)
```

Original DataFrame:

	0
Reference Number	0
Grid Ref: Easting	0
Grid Ref: Northing	0
Number of Vehicles	0
Accident Date	0
Time (24hr)	0
1st Road Class	0
1st Road Class & No	0
Road Surface	0
Lighting Conditions	0
Weather Conditions	0
Local Authority	0
Vehicle Number	0
Type of Vehicle	0
Casualty Class	0
Casualty Severity	0
Sex of Casualty	0
Age of Casualty	0

Normalized DataFrame

```
In [45]: # print("\nNormalized DataFrame:")  
print(normalized_df2)
```

```
      0  
0  0.0  
1  0.0  
2  0.0  
3  0.0  
4  0.0  
5  0.0  
6  0.0  
7  0.0  
8  0.0  
9  0.0  
10 0.0  
11 0.0  
12 0.0  
13 0.0  
14 0.0  
15 0.0  
16 0.0  
17 0.0
```

Check for Duplicate Columns and Rows

```
In [46]: import pandas as pd

# Load your data from the CSV file
data2 = pd.read_csv('Traffic accidents_2019_Leeds.csv')

# Identify duplicate columns
duplicate_columns = []
seen_columns = set()
for column in data1.columns:
    if column in seen_columns:
        duplicate_columns.append(column)
    else:
        seen_columns.add(column)

# Identify duplicate rows
duplicate_rows = data1[data1.duplicated()]

if duplicate_columns:
    print("Duplicate columns found:")
    for column in duplicate_columns:
        print(column)
else:
    print("No duplicate columns found.")

if not duplicate_rows.empty:
    print("\nDuplicate rows found:")
    print(duplicate_rows)
else:
    print("No duplicate rows found.")
```

No duplicate columns found.

Duplicate rows found:

	Accident Fields_Reference Number	Grid Ref: Easting	Grid Ref: Northing
4	51B0349	430076	433872
230	5240945	432285	433647
335	5200005	419422	443129
336	5200005	419422	443129
390	53A0189	428111	426283
394	53A0824	430676	434874
582	54I1128	429654	431554
1039	56U0266	423867	437663
1043	56U0266	423867	437663
1886	5C90576	428937	429842
1993	5CV0814	432096	427519

	Number of Vehicles	Accident Date	Time (24hr)	1st Road Class	\
4	1	11/01/2018	855	6	
230	2	04/02/2018	435	3	
335	1	24/02/2018	5	6	
336	1	24/02/2018	5	6	
390	2	10/03/2018	215	3	
394	2	08/03/2018	2300	3	
582	1	06/04/2018	1535	6	
1039	2	30/06/2018	238	3	
1043	2	30/06/2018	238	3	
1886	2	09/12/2018	1047	6	
1993	2	31/12/2018	1345	1	

	1st Road Class & No	Road Surface	Lighting Conditions	...	\
4	U	2	1	...	
230	A64	1	4	...	
335	U	1	7	...	
336	U	1	7	...	
390	A650	2	4	...	
394	A61	1	4	...	
582	U	1	1	...	
1039	A6120	1	4	...	
1043	A6120	1	4	...	
1886	U	1	1	...	
1993	M1	2	1	...	

	Local Authority	Vehicle Fields_Reference Number	Vehicle Number	\
4	E08000035	51B0349	1	
230	E08000035	5240945	2	
335	E08000035	5200005	1	
336	E08000035	5200005	1	
390	E08000035	53A0189	1	
394	E08000035	53A0824	2	
582	E08000035	54I1128	1	
1039	E08000035	56U0266	1	
1043	E08000035	56U0266	1	
1886	E08000035	5C90576	2	
1993	E08000035	5CV0814	2	

Type of Vehicle	Casualty Fields_Reference Number	Casualty Veh No	\
-----------------	----------------------------------	-----------------	---

4	11	51B0349	1
230	8	5240945	2
335	9	5200005	1
336	9	5200005	1
390	8	53A0189	1
394	9	53A0824	2
582	11	54I1128	1
1039	9	56U0266	1
1043	9	56U0266	1
1886	9	5C90576	2
1993	9	5CV0814	2

	Casualty Class	Casualty Severity	Sex of Casualty	Age of Casualty
4	2	3	2	51
230	2	3	2	17
335	2	3	1	17
336	2	3	1	17
390	2	2	2	16
394	2	3	1	25
582	2	3	2	31
1039	2	2	2	17
1043	2	1	1	19
1886	2	3	1	52
1993	2	3	2	9

[11 rows x 21 columns]

Remove Duplicate Rows and Columns

```
In [47]: import pandas as pd

# Load your data from the CSV file
data2 = pd.read_csv('Traffic accidents_2019_Leeds.csv')

# Remove duplicate rows
data2 = data2.drop_duplicates()

# Remove duplicate columns
data2 = data2.loc[:, ~data2.columns.duplicated()]

# Save the cleaned data to a new CSV file
data2.to_csv('cleaned_data2.csv', index=False)
```

Generate Descriptive Analysis

Check whether the dataset is imbalanced

```
In [48]: import pandas as pd

file_path1 = 'RTC 2018_Leeds.csv'
data1 = pd.read_csv(file_path1)

df1 = pd.DataFrame(data1)

# Identify columns with imbalanced data (categorical columns with significantly
imbalanced_columns = []

for column in df1.columns:
    if df1[column].dtype == 'object':
        value_counts = df1[column].value_counts()
        if len(value_counts) <= 1 or value_counts.max() / value_counts.min() >= 10:
            imbalanced_columns.append(column)

print("Imbalanced Columns:")
print(imbalanced_columns)
```

```
Imbalanced Columns:
['Accident Date', '1st Road Class & No', 'Local Authority']
```

Mean , Median or Mode

```
In [50]: import pandas as pd
import numpy as np
from statistics import mode

# Load your data from the CSV file
data2 = pd.read_csv('Traffic accidents_2019_Leeds.csv')

# Filter only the numeric columns
numeric_columns = data2.select_dtypes(include=[np.number])

# Calculate and display the mean, median, and mode for each numeric column
for column in numeric_columns.columns:
    mean = np.mean(numeric_columns[column])
    median = np.median(numeric_columns[column])

    # Flatten the column into a 1D array for mode calculation
    flat_data2 = numeric_columns[column].values.flatten()
    try:
        mode_value = mode(flat_data2)
    except StatisticsError:
        mode_value = "No unique mode found"

    print(f"Column: {column}")
    print(f"Mean: {mean}")
    print(f"Median: {median}")
    print(f"Mode: {mode_value}")
    print()
```

Column: Grid Ref: Easting
Mean: 430273.6046145779
Median: 430096.0
Mode: 430846

Column: Grid Ref: Northing
Mean: 434036.86156266386
Median: 433982.0
Mode: 434032

Column: Number of Vehicles
Mean: 1.9690613529103305
Median: 2.0
Mode: 2

Column: Time (24hr)
Mean: 1385.4950183534347
Median: 1500.0
Mode: 1630

Column: 1st Road Class
Mean: 4.399580492920818
Median: 6.0
Mode: 6

Column: Road Surface
Mean: 1.3219716832721553
Median: 1.0
Mode: 1

Column: Lighting Conditions
Mean: 1.911903513371788
Median: 1.0
Mode: 1

Column: Weather Conditions
Mean: 1.241740954378605
Median: 1.0
Mode: 1

Column: Vehicle Number
Mean: 1.5579444153120083
Median: 2.0
Mode: 1

Column: Type of Vehicle
Mean: 8.344520188778185
Median: 9.0
Mode: 9

Column: Casualty Class
Mean: 1.5673833245936024
Median: 1.0
Mode: 1

Column: Casualty Severity
Mean: 2.8017829050865233

Median: 3.0

Mode: 3

Column: Sex of Casualty

Mean: 1.3974829575249081

Median: 1.0

Mode: 1

Column: Age of Casualty

Mean: 35.79811221814368

Median: 32.0

Mode: 27

Range, Variance, Standard Deviation or Interquartile Range (IQR)

```
In [51]: import pandas as pd
import numpy as np

# Load your data from the CSV file
data2 = pd.read_csv('Traffic accidents_2019_Leeds.csv')

# Select only numeric columns
numeric_data = data2.select_dtypes(include=[np.number])

# Calculate measures of dispersion for all numeric columns
dispersion_metrics = {
    'Column': [],
    'Range': [],
    'Variance': [],
    'Standard Deviation': [],
    'Interquartile Range (IQR)': []
}

for column in numeric_data.columns:
    dispersion_metrics['Column'].append(column)
    dispersion_metrics['Range'].append(np.ptp(numeric_data[column]))
    dispersion_metrics['Variance'].append(np.var(numeric_data[column]))
    dispersion_metrics['Standard Deviation'].append(np.std(numeric_data[column]))
    q1 = numeric_data[column].quantile(0.25)
    q3 = numeric_data[column].quantile(0.75)
    dispersion_metrics['Interquartile Range (IQR)'].append(q3 - q1)

# Create a summary DataFrame
dispersion_df = pd.DataFrame(dispersion_metrics)

# Print the results
print(dispersion_df)
```

	Column	Range	Variance	Standard Deviation \
0	Grid Ref: Easting	30734	2.654893e+07	5152.565497
1	Grid Ref: Northing	26442	2.353500e+07	4851.288751
2	Number of Vehicles	5	6.707785e-01	0.819011
3	Time (24hr)	2355	2.671710e+05	516.885823
4	1st Road Class	5	3.247782e+00	1.802160
5	Road Surface	8	3.651334e-01	0.604263
6	Lighting Conditions	6	2.431673e+00	1.559382
7	Weather Conditions	8	6.646866e-01	0.815283
8	Vehicle Number	4	3.840310e-01	0.619702
9	Type of Vehicle	96	3.430239e+01	5.856824
10	Casualty Class	2	5.884065e-01	0.767077
11	Casualty Severity	2	1.820000e-01	0.426615
12	Sex of Casualty	1	2.394903e-01	0.489377
13	Age of Casualty	94	3.361915e+02	18.335527

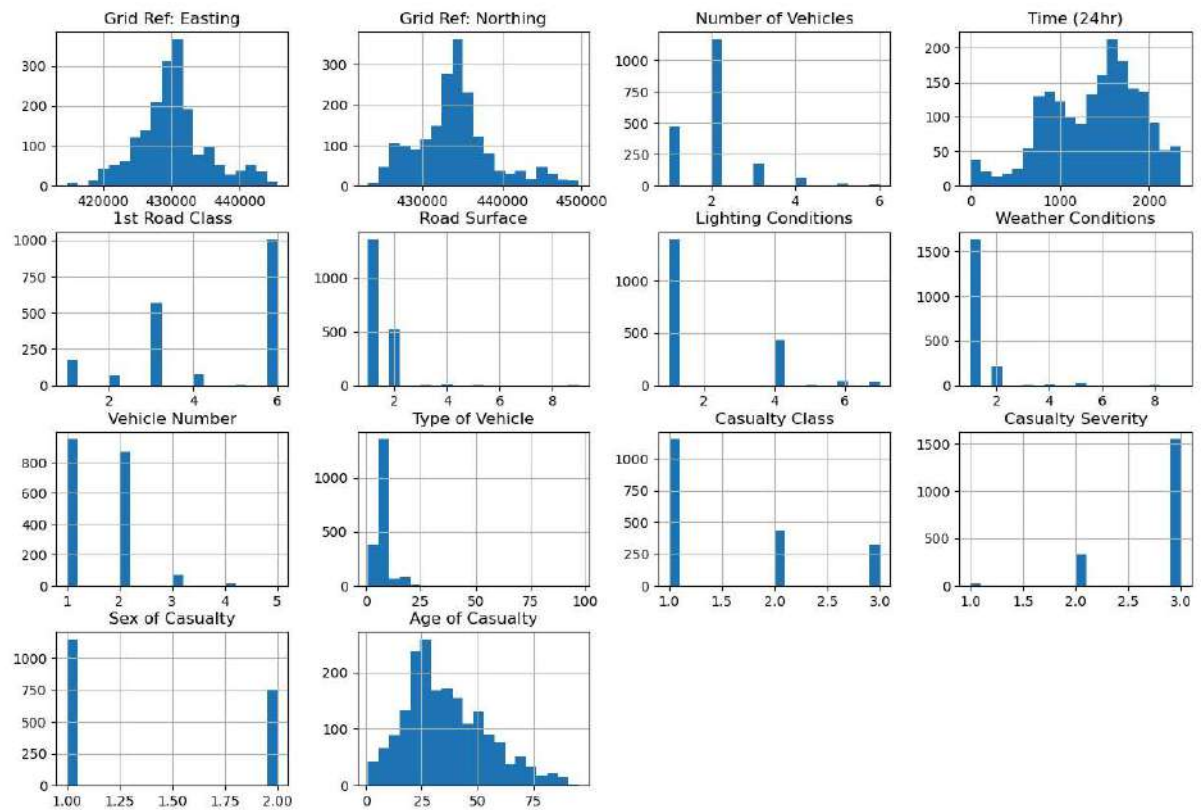
	Interquartile Range (IQR)
0	4845.0
1	4749.5
2	0.0
3	756.0
4	3.0
5	1.0
6	3.0
7	0.0
8	1.0
9	0.0
10	1.0
11	0.0
12	1.0
13	24.5

Frequency and Data Distributions using Histogram

```
In [28]: import matplotlib.pyplot as plt
import seaborn as sns

# Load your dataset into a DataFrame
file_path2 = 'Traffic accidents_2019_Leeds.csv'
data2 = pd.read_csv(file_path2)

# Histograms for all numeric columns
data2.hist(bins=20, figsize=(15, 10))
plt.show()
```



Summary statistics

```
In [3]: # Summary statistics
summary = data2.describe()
print(summary)
```

	Grid Ref: Easting	Grid Ref: Northing	Number of Vehicles	Time (24hr)
\				
count	1907.000000	1907.000000	1907.000000	1907.000000
mean	430273.604615	434036.861563	1.969061	1385.495018
std	5153.916990	4852.561221	0.819226	517.021399
min	414747.000000	423157.000000	1.000000	0.000000
25%	427396.000000	431197.500000	2.000000	1000.000000
50%	430096.000000	433982.000000	2.000000	1500.000000
75%	432241.000000	435947.000000	2.000000	1756.000000
max	445481.000000	449599.000000	6.000000	2355.000000

	1st Road Class	Road Surface	Lighting Conditions	Weather Conditions
\				
count	1907.000000	1907.000000	1907.000000	1907.000000
mean	4.399580	1.321972	1.911904	1.241741
std	1.802633	0.604421	1.559791	0.815497
min	1.000000	1.000000	1.000000	1.000000
25%	3.000000	1.000000	1.000000	1.000000
50%	6.000000	1.000000	1.000000	1.000000
75%	6.000000	2.000000	4.000000	1.000000
max	6.000000	9.000000	7.000000	9.000000

	Vehicle Number	Type of Vehicle	Casualty Class	Casualty Severity	\
count	1907.000000	1907.000000	1907.000000	1907.000000	
mean	1.557944	8.34452	1.567383	2.801783	
std	0.619865	5.85836	0.767278	0.426726	
min	1.000000	1.000000	1.000000	1.000000	
25%	1.000000	9.000000	1.000000	3.000000	
50%	2.000000	9.000000	1.000000	3.000000	
75%	2.000000	9.000000	2.000000	3.000000	
max	5.000000	97.000000	3.000000	3.000000	

	Sex of Casualty	Age of Casualty
count	1907.000000	1907.000000
mean	1.397483	35.798112
std	0.489506	18.340336
min	1.000000	1.000000
25%	1.000000	22.500000
50%	1.000000	32.000000
75%	2.000000	47.000000
max	2.000000	95.000000

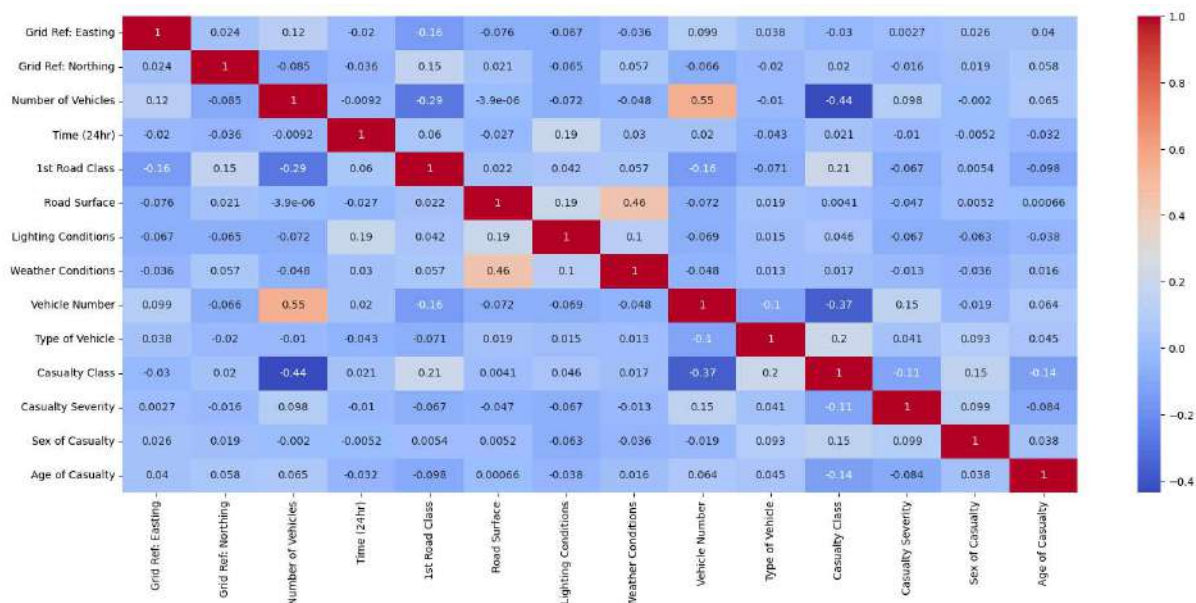
Correlation heatmap for numeric columns

```
In [7]: import seaborn as sns # Import Seaborn Library
import matplotlib.pyplot as plt # Import Matplotlib Library

correlation_matrix = data2.corr()
plt.figure(figsize=(20, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.show()
```

C:\Users\lenovo\AppData\Local\Temp\ipykernel_4388\680900661.py:4: FutureWarning: The default value of numeric_only in DataFrame.corr is deprecated. In a future version, it will default to False. Select only valid columns or specify the value of numeric_only to silence this warning.

```
correlation_matrix = data2.corr()
```

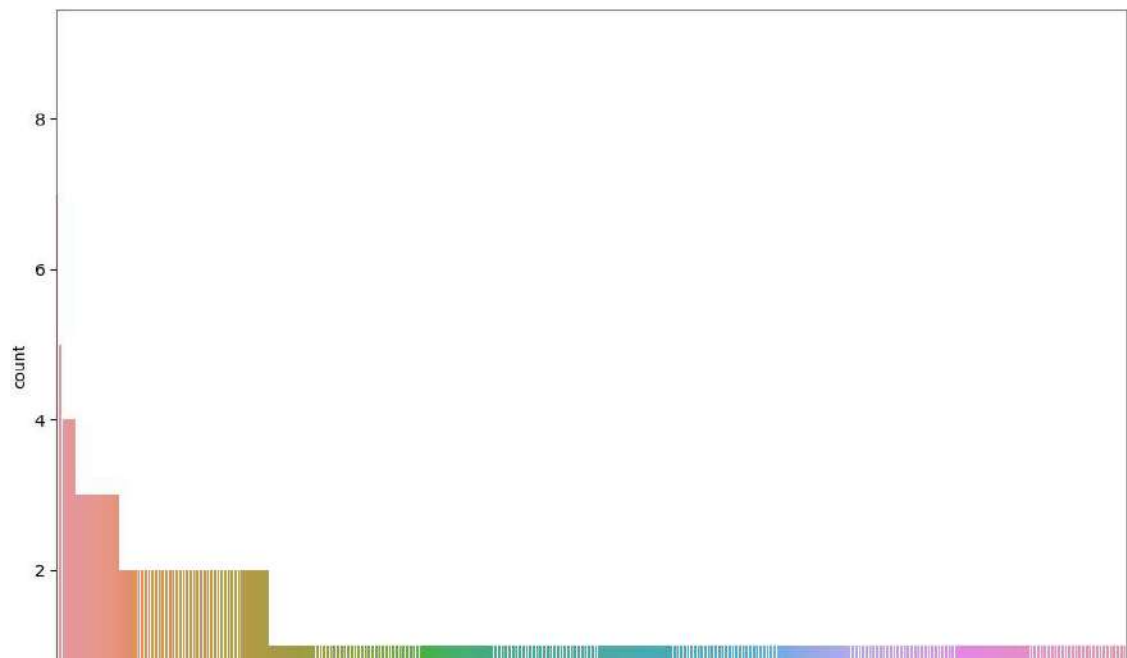


Frequency distribution for categorical columns

```
In [9]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

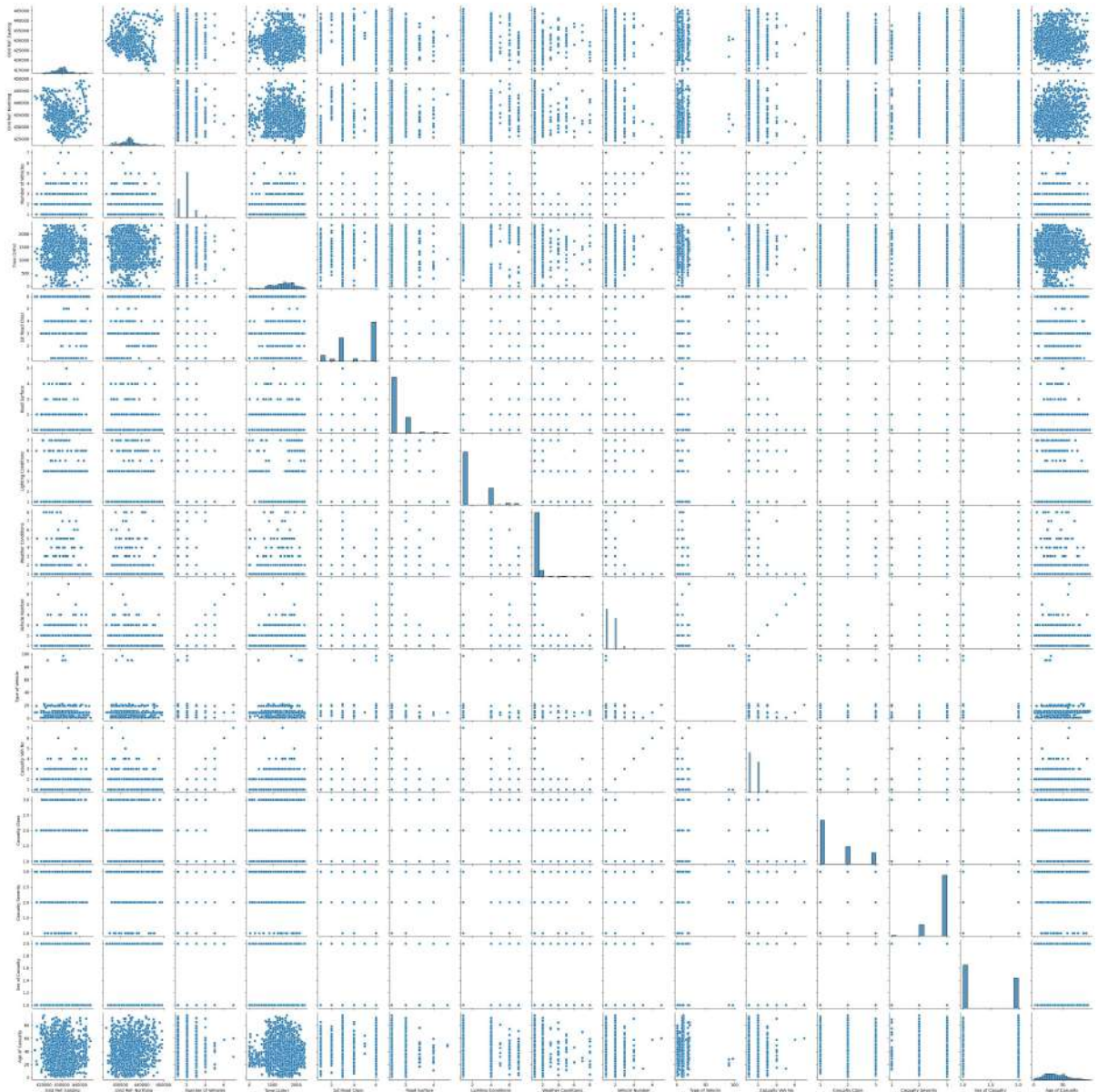
file_path2 = 'RTC 2018_Leeds.csv'
data2 = pd.read_csv(file_path2)

# Frequency distribution for categorical columns
categorical_columns = data2.select_dtypes(include='object')
for column in categorical_columns:
    plt.figure(figsize=(12, 8))
    sns.countplot(data=data2, x=column, order=data2[column].value_counts().index)
    plt.xticks(rotation=45)
    plt.show()
```



Pairwise scatter plot matrix for numeric columns

```
In [11]: # Pairwise scatter plot matrix for numeric columns
sns.pairplot(data2, kind='scatter')
plt.show()
```



Find missing values

```
In [12]: # Check for missing values
missing_values = data2.isnull().sum()
print("Missing Values:")
print(missing_values)
```

```
Missing Values:
Accident Fields_Reference Number    0
Grid Ref: Easting                   0
Grid Ref: Northing                   0
Number of Vehicles                   0
Accident Date                        0
Time (24hr)                         0
1st Road Class                       0
1st Road Class & No                  0
Road Surface                         0
Lighting Conditions                  0
Weather Conditions                   0
Local Authority                      0
Vehicle Fields_Reference Number      0
Vehicle Number                       0
Type of Vehicle                      0
Casualty Fields_Reference Number     0
Casualty Veh No                      0
Casualty Class                       0
Casualty Severity                    0
Sex of Casualty                      0
Age of Casualty                      0
dtype: int64
```

Check data types

```
In [13]: # Check data types
file_path2 = 'RTC 2018_Leeds.csv'
data2 = pd.read_csv(file_path2)

data2_types = data2.dtypes
print("\nData Types:")
print(data2_types)
```

```
Data Types:
Accident Fields_Reference Number    object
Grid Ref: Easting                    int64
Grid Ref: Northing                   int64
Number of Vehicles                   int64
Accident Date                        object
Time (24hr)                          int64
1st Road Class                       int64
1st Road Class & No                  object
Road Surface                         int64
Lighting Conditions                  int64
Weather Conditions                   int64
Local Authority                      object
Vehicle Fields_Reference Number      object
Vehicle Number                       int64
Type of Vehicle                      int64
Casualty Fields_Reference Number      object
Casualty Veh No                      int64
Casualty Class                       int64
Casualty Severity                    int64
Sex of Casualty                      int64
Age of Casualty                      int64
dtype: object
```

Check the number of unique values for each column

```
In [14]: unique_values = data2.nunique()  
print("\nUnique Values:")  
print(unique_values)
```

```
Unique Values:  
Accident Fields_Reference Number    1548  
Grid Ref: Easting                   1432  
Grid Ref: Northing                  1422  
Number of Vehicles                    7  
Accident Date                       354  
Time (24hr)                         621  
1st Road Class                       6  
1st Road Class & No                  50  
Road Surface                         5  
Lighting Conditions                 5  
Weather Conditions                  8  
Local Authority                     1  
Vehicle Fields_Reference Number    1548  
Vehicle Number                      7  
Type of Vehicle                     16  
Casualty Fields_Reference Number    1548  
Casualty Veh No                     7  
Casualty Class                      3  
Casualty Severity                   3  
Sex of Casualty                     2  
Age of Casualty                     94  
dtype: int64
```

Check for duplicate rows

```
In [15]: duplicate_rows = data2.duplicated().sum()  
print("\nDuplicate Rows:")  
print(duplicate_rows)
```

```
Duplicate Rows:  
11
```

Check for outliers in file

```
In [31]: import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
data2 = pd.read_csv('Traffic accidents_2019_Leeds.csv')

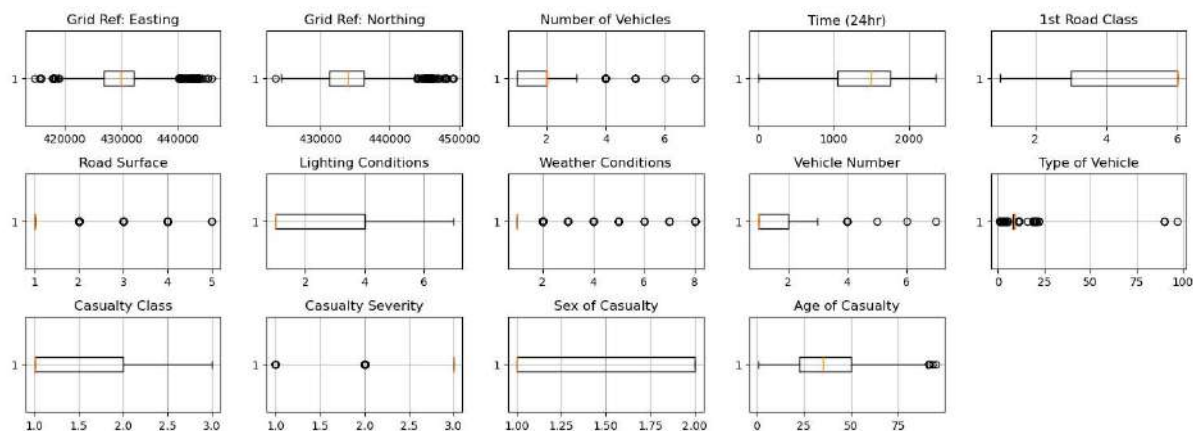
# Select columns with numeric data types (excluding non-numeric columns)
numeric_columns = data2.select_dtypes(include=['int64', 'float64'])

# Set the figure size
plt.figure(figsize=(15, 7))

# Create box plots for each numeric column
for i, column in enumerate(numeric_columns.columns):
    plt.subplot(4, 5, i + 1)
    plt.boxplot(data2[column], vert=False)
    plt.title(column)
    plt.grid(True)

# Adjust the layout
plt.tight_layout()

# Show the box plots
plt.show()
```



Remove Outliers

```
In [26]: import pandas as pd
import numpy as np

def remove_outliers(file_path, threshold=1.5):
    # Read the CSV file into a DataFrame
    data2 = pd.read_csv(file_path)

    # Create a copy of the original data
    origina2_data = data2.copy()

    # Detect outliers using the IQR method
    Q1 = data2.quantile(0.25, numeric_only=True)
    Q3 = data2.quantile(0.75, numeric_only=True)

    IQR = Q3 - Q1

    lower_bound = Q1 - threshold * IQR
    upper_bound = Q3 + threshold * IQR

    # Align DataFrames and create a boolean mask to identify outliers
    left, right = data2.align(lower_bound, axis=1, copy=False)
    outliers_mask = ((left < right) | (left > right))

    # Remove rows that contain outliers
    data_no_outliers = data2[~outliers_mask.any(axis=1)]

    return origina2_data, data_no_outliers

# Example usage
file_path = 'Traffic accidents_2019_Leeds.csv'
original_data, data_no_outliers = remove_outliers(file_path)

# Save the data without outliers to a new CSV file
data_no_outliers.to_csv('data_no_outliers.csv', index=False)
# print(data_no_outliers)
```

Data adequacy assessment summary

```
In [27]: if missing_values.sum() == 0 and duplicate_rows == 0:
    print("\nData appears to be adequate.")
else:
    print("\nData has issues that need to be addressed.")
```

Data has issues that need to be addressed.

Models

linear Regression

```
In [5]: # Import necessary libraries
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Load your dataset (replace 'your_dataset.csv' with the actual dataset file)
data = pd.read_csv('Traffic accidents_2019_Leeds.csv')

# Select the features and target variable
X = data[['Number of Vehicles', '1st Road Class', 'Road Surface', 'Lighting Condition']]
y = data['Casualty Severity']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create a Linear Regression model
model = LinearRegression()

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# Print the model coefficients and evaluation metrics
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
print("Mean Squared Error:", mse)
print("R-squared:", r2)
```

```
Coefficients: [ 0.04269876 -0.01315217 -0.04068989 -0.01648262  0.01003534]
Intercept: 2.847079774734122
Mean Squared Error: 0.18666500926387025
R-squared: -0.003607266195829295
```

Random Forest

```
In [6]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load your dataset (replace 'RTC 2018_Leeds.csv' with the actual dataset file)
data = pd.read_csv('Traffic accidents_2019_Leeds.csv')

# Select the features and target variable
X = data[['Number of Vehicles', '1st Road Class', 'Road Surface', 'Lighting Con
y = data['Casualty Severity']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create a Random Forest classifier
model = RandomForestClassifier(n_estimators=100, random_state=42) # You can ad

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

# Print the model's accuracy and classification report
print("Accuracy:", accuracy)
print("Classification Report:\n", report)
```

Accuracy: 0.8167539267015707

Classification Report:

	precision	recall	f1-score	support
1	0.00	0.00	0.00	6
2	0.17	0.02	0.03	61
3	0.83	0.99	0.90	315
accuracy			0.82	382
macro avg	0.33	0.33	0.31	382
weighted avg	0.71	0.82	0.75	382

```
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and bei
ng set to 0.0 in labels with no predicted samples. Use `zero_division` parame
ter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and bei
ng set to 0.0 in labels with no predicted samples. Use `zero_division` parame
ter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and bei
ng set to 0.0 in labels with no predicted samples. Use `zero_division` parame
ter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
```

SVM Model

```
In [7]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# Load your dataset (replace 'RTC 2018_Leeds.csv' with the actual dataset file)
data = pd.read_csv('Traffic accidents_2019_Leeds.csv')

# Select the features and target variable
X = data[['Number of Vehicles', '1st Road Class', 'Road Surface', 'Lighting Con
y = data['Casualty Severity']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create an SVM classifier
model = SVC(kernel='linear', random_state=42) # You can choose the kernel type

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

# Print the model's accuracy and classification report
print("Accuracy:", accuracy)
print("Classification Report:\n", report)
```

Accuracy: 0.824607329842932

Classification Report:

	precision	recall	f1-score	support
1	0.00	0.00	0.00	6
2	0.00	0.00	0.00	61
3	0.82	1.00	0.90	315
accuracy			0.82	382
macro avg	0.27	0.33	0.30	382
weighted avg	0.68	0.82	0.75	382

```
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and bei
ng set to 0.0 in labels with no predicted samples. Use `zero_division` parame
ter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and bei
ng set to 0.0 in labels with no predicted samples. Use `zero_division` parame
ter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and bei
ng set to 0.0 in labels with no predicted samples. Use `zero_division` parame
ter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
```

Decision Trees

```
In [8]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load your dataset (replace 'RTC 2018_Leeds.csv' with the actual dataset file)
data = pd.read_csv('Traffic accidents_2019_Leeds.csv')

# Select the features and target variable
X = data[['Number of Vehicles', '1st Road Class', 'Road Surface', 'Lighting Con
y = data['Casualty Severity']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create a Decision Tree classifier
model = DecisionTreeClassifier(random_state=42)

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

# Print the model's accuracy and classification report
print("Accuracy:", accuracy)
print("Classification Report:\n", report)
```

Accuracy: 0.8167539267015707

Classification Report:

	precision	recall	f1-score	support
1	0.00	0.00	0.00	6
2	0.25	0.03	0.06	61
3	0.83	0.98	0.90	315
accuracy			0.82	382
macro avg	0.36	0.34	0.32	382
weighted avg	0.72	0.82	0.75	382

```
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and bei
ng set to 0.0 in labels with no predicted samples. Use `zero_division` parame
ter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and bei
ng set to 0.0 in labels with no predicted samples. Use `zero_division` parame
ter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and bei
ng set to 0.0 in labels with no predicted samples. Use `zero_division` parame
ter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
```

Evaluation Metrics For Dataset2

Accuracy Metrics

```
In [12]: # Import necessary Libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load your dataset
data = pd.read_csv('Traffic accidents_2019_Leeds.csv')

# Select the features and target variable
X = data[['Number of Vehicles', '1st Road Class', 'Road Surface', 'Lighting Condition']]
y = data['Casualty Severity']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create a Decision Tree classifier
model = DecisionTreeClassifier(random_state=42)

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

# Print the accuracy
print("Accuracy:", accuracy)
```

Accuracy: 0.8167539267015707

Precision and Recall Metrics

```
In [13]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import precision_score, recall_score

# Load your dataset
data = pd.read_csv('Traffic accidents_2019_Leeds.csv')

# Select the features and target variable
X = data[['Number of Vehicles', '1st Road Class', 'Road Surface', 'Lighting Con
y = data['Casualty Severity']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create a Decision Tree classifier
model = DecisionTreeClassifier(random_state=42)

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Calculate Precision and Recall
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')

# Print the Precision and Recall
print("Precision:", precision)
print("Recall:", recall)
```

Precision: 0.723419520116471

Recall: 0.8167539267015707

C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.p
y:1344: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0
in labels with no predicted samples. Use `zero\_division` parameter to control
this behavior.

_warn_prf(average, modifier, msg_start, len(result))

Mean Absolute Error (MAE) or Mean Squared Error (MSE) Metrics

```
In [14]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error

# Load your dataset
data = pd.read_csv('Traffic accidents_2019_Leeds.csv')

# Select the features and target variable
X = data[['Number of Vehicles', '1st Road Class', 'Road Surface', 'Lighting Con
y = data['Casualty Severity']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create a Decision Tree Regressor (for regression)
model = DecisionTreeRegressor(random_state=42)

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Calculate Mean Absolute Error (MAE)
mae = mean_absolute_error(y_test, y_pred)

# Calculate Mean Squared Error (MSE)
mse = mean_squared_error(y_test, y_pred)

# Print MAE and MSE
print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
```

Mean Absolute Error (MAE): 0.2997341664136071

Mean Squared Error (MSE): 0.19697971836828407

Log Loss (Logarithmic Loss) Metrics

```
In [15]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import log_loss

# Load your dataset
data = pd.read_csv('Traffic accidents_2019_Leeds.csv')

# Select the features and target variable
X = data[['Number of Vehicles', '1st Road Class', 'Road Surface', 'Lighting Con
y = data['Casualty Severity']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create a Random Forest classifier
model = RandomForestClassifier(n_estimators=100, random_state=42)

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data (probabilities)
y_prob = model.predict_proba(X_test)

# Calculate Log Loss
logloss = log_loss(y_test, y_prob)

# Print Log Loss
print("Log Loss:", logloss)
```

Log Loss: 0.9155497892504995

Root Mean Squared Error (RMSE) Metrics

```
In [16]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error
import numpy as np

# Load your dataset
data = pd.read_csv('Traffic accidents_2019_Leeds.csv')

# Select the features and target variable
X = data[['Number of Vehicles', '1st Road Class', 'Road Surface', 'Lighting Con
y = data['Casualty Severity']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create a Decision Tree Regressor (for regression)
model = DecisionTreeRegressor(random_state=42)

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Calculate Mean Squared Error (MSE)
mse = mean_squared_error(y_test, y_pred)

# Calculate Root Mean Squared Error (RMSE)
rmse = np.sqrt(mse)

# Print RMSE
print("Root Mean Squared Error (RMSE):", rmse)
```

Root Mean Squared Error (RMSE): 0.4438239722776183

DataSet 3

In []:

In []:

Missing Data

```
In [31]: # Print missing data status
print("\nMissing Data Status:")
print(missing_data3)
```

```
Missing Data Status:
0
STATE/UT      False
YEAR          False
0-3 hrs. (Night) False
3-6 hrs. (Night) False
6-9 hrs (Day)  False
9-12 hrs (Day) False
12-15 hrs (Day) False
15-18 hrs (Day) False
18-21 hrs (Night) False
21-24 hrs (Night) False
Total         False
```

Print missing data counts

```
In [32]: print("\nMissing Data Counts:")
print(missing_count3)
```

```
Missing Data Counts:
0    0
dtype: int64
```

Drop Rows with Missing Values

```
In [34]: df3_dropped3 = df3.dropna()
```

Fill Missing Values with a Specific Value

```
In [35]: df3_filled3 = df3.fillna(0)
```

Replace Missing Values with the Mean of the Column

```
In [36]: mean_fill3 = df3.fillna(df3.mean())
```

Print DataFrames after handling missing data

```
In [37]: print("\nDataFrame after dropping rows with missing data:")  
print(df3_dropped3)
```

DataFrame after dropping rows with missing data:

	0
STATE/UT	0
YEAR	0
0-3 hrs. (Night)	0
3-6 hrs. (Night)	0
6-9 hrs (Day)	0
9-12 hrs (Day)	0
12-15 hrs (Day)	0
15-18 hrs (Day)	0
18-21 hrs (Night)	0
21-24 hrs (Night)	0
Total	0

```
In [38]: # print("\nDataFrame after filling missing data with 0:")  
print(df3_filled3)
```

	0
STATE/UT	0
YEAR	0
0-3 hrs. (Night)	0
3-6 hrs. (Night)	0
6-9 hrs (Day)	0
9-12 hrs (Day)	0
12-15 hrs (Day)	0
15-18 hrs (Day)	0
18-21 hrs (Night)	0
21-24 hrs (Night)	0
Total	0

```
In [39]: # print("\nDataFrame after filling missing data with column mean:")  
print(mean_fill3)
```

	0
STATE/UT	0
YEAR	0
0-3 hrs. (Night)	0
3-6 hrs. (Night)	0
6-9 hrs (Day)	0
9-12 hrs (Day)	0
12-15 hrs (Day)	0
15-18 hrs (Day)	0
18-21 hrs (Night)	0
21-24 hrs (Night)	0
Total	0

```
In [40]: from sklearn.preprocessing import MinMaxScaler
```

```
In [42]: # Initialize the MinMaxScaler
scaler = MinMaxScaler()

# Fit and transform the DataFrame to normalize the data
normalized_data3 = scaler.fit_transform(df3)
```

```
In [43]: # Create a new DataFrame with the normalized values
normalized_df3 = pd.DataFrame(normalized_data3, columns=df3.columns)
```

Original DataFrame

```
In [44]: print("Original DataFrame:")
print(df3)
```

```
Original DataFrame:
0
STATE/UT      0
YEAR          0
0-3 hrs. (Night) 0
3-6 hrs. (Night) 0
6-9 hrs (Day)    0
9-12 hrs (Day)   0
12-15 hrs (Day)  0
15-18 hrs (Day)  0
18-21 hrs (Night) 0
21-24 hrs (Night) 0
Total          0
```

Normalized DataFrame

```
In [45]: # print("\nNormalized DataFrame:")
print(normalized_df3)
```

```
0
0  0.0
1  0.0
2  0.0
3  0.0
4  0.0
5  0.0
6  0.0
7  0.0
8  0.0
9  0.0
10 0.0
```

Check for Duplicate Columns and Rows

```
In [46]: import pandas as pd

# Load your data from the CSV file
data3 = pd.read_csv('only_road_accidents_data3.csv')

# Identify duplicate columns
duplicate_columns = []
seen_columns = set()
for column in data3.columns:
    if column in seen_columns:
        duplicate_columns.append(column)
    else:
        seen_columns.add(column)

# Identify duplicate rows
duplicate_rows = data3[data3.duplicated()]

if duplicate_columns:
    print("Duplicate columns found:")
    for column in duplicate_columns:
        print(column)
else:
    print("No duplicate columns found.")

if not duplicate_rows.empty:
    print("\nDuplicate rows found:")
    print(duplicate_rows)
else:
    print("No duplicate rows found.")
```

No duplicate columns found.

No duplicate rows found.

Check whether the dataset is imbalanced

```
In [47]: import pandas as pd

file_path3 = 'only_road_accidents_data3.csv'
data3 = pd.read_csv(file_path3)

df3 = pd.DataFrame(data3)

# Identify columns with imbalanced data (categorical columns with significantly
imbalanced_columns = []

for column in df3.columns:
    if df3[column].dtype == 'object':
        value_counts = df3[column].value_counts()
        if len(value_counts) <= 1 or value_counts.max() / value_counts.min() >= 10:
            imbalanced_columns.append(column)

print("Imbalanced Columns:")
print(imbalanced_columns)
```

Imbalanced Columns:
[]

Mean , Median or Mode

```
In [48]: import pandas as pd
import numpy as np
from statistics import mode

# Load your data from the CSV file
data3 = pd.read_csv('only_road_accidents_data3.csv')

# Filter only the numeric columns
numeric_columns = data3.select_dtypes(include=[np.number])

# Calculate and display the mean, median, and mode for each numeric column
for column in numeric_columns.columns:
    mean = np.mean(numeric_columns[column])
    median = np.median(numeric_columns[column])

    # Flatten the column into a 1D array for mode calculation
    flat_data3 = numeric_columns[column].values.flatten()
    try:
        mode_value = mode(flat_data3)
    except StatisticsError:
        mode_value = "No unique mode found"

    print(f"Column: {column}")
    print(f"Mean: {mean}")
    print(f"Median: {median}")
    print(f"Mode: {mode_value}")
    print()
```

Column: YEAR
Mean: 2007.5
Median: 2007.5
Mode: 2001

Column: 0-3 hrs. (Night)
Mean: 796.3204081632653
Median: 200.0
Mode: 0

Column: 3-6 hrs. (Night)
Mean: 969.2367346938776
Median: 251.5
Mode: 0

Column: 6-9 hrs (Day)
Mean: 1371.1510204081633
Median: 498.5
Mode: 0

Column: 9-12 hrs (Day)
Mean: 1753.9673469387756
Median: 678.0
Mode: 0

Column: 12-15 hrs (Day)
Mean: 1681.8142857142857
Median: 679.5
Mode: 0

Column: 15-18 hrs (Day)
Mean: 1850.2836734693878
Median: 669.0
Mode: 0

Column: 18-21 hrs (Night)
Mean: 1782.9183673469388
Median: 517.0
Mode: 0

Column: 21-24 hrs (Night)
Mean: 1228.8102040816327
Median: 343.5
Mode: 0

Column: Total
Mean: 11434.502040816327
Median: 4174.5
Mode: 0

Range, Variance, Standard Deviation or Interquartile Range (IQR)

```
In [49]: import pandas as pd
import numpy as np

# Load your data from the CSV file
data3 = pd.read_csv('only_road_accidents_data3.csv')

# Select only numeric columns
numeric_data = data3.select_dtypes(include=[np.number])

# Calculate measures of dispersion for all numeric columns
dispersion_metrics = {
    'Column': [],
    'Range': [],
    'Variance': [],
    'Standard Deviation': [],
    'Interquartile Range (IQR)': []
}

for column in numeric_data.columns:
    dispersion_metrics['Column'].append(column)
    dispersion_metrics['Range'].append(np.ptp(numeric_data[column]))
    dispersion_metrics['Variance'].append(np.var(numeric_data[column]))
    dispersion_metrics['Standard Deviation'].append(np.std(numeric_data[column]))
    q1 = numeric_data[column].quantile(0.25)
    q3 = numeric_data[column].quantile(0.75)
    dispersion_metrics['Interquartile Range (IQR)'].append(q3 - q1)

# Create a summary DataFrame
dispersion_df = pd.DataFrame(dispersion_metrics)

# Print the results
print(dispersion_df)
```

	Column	Range	Variance	Standard Deviation \
0	YEAR	13	1.625000e+01	4.031129
1	0-3 hrs. (Night)	6020	1.463526e+06	1209.762981
2	3-6 hrs. (Night)	6545	2.000920e+06	1414.538881
3	6-9 hrs (Day)	8241	3.468118e+06	1862.288362
4	9-12 hrs (Day)	14568	5.751088e+06	2398.142696
5	12-15 hrs (Day)	11089	5.400403e+06	2323.876727
6	15-18 hrs (Day)	13789	6.975230e+06	2641.066164
7	18-21 hrs (Night)	13927	6.681247e+06	2584.810847
8	21-24 hrs (Night)	10364	3.364907e+06	1834.368175
9	Total	75480	2.538416e+08	15932.406986

Interquartile Range (IQR)

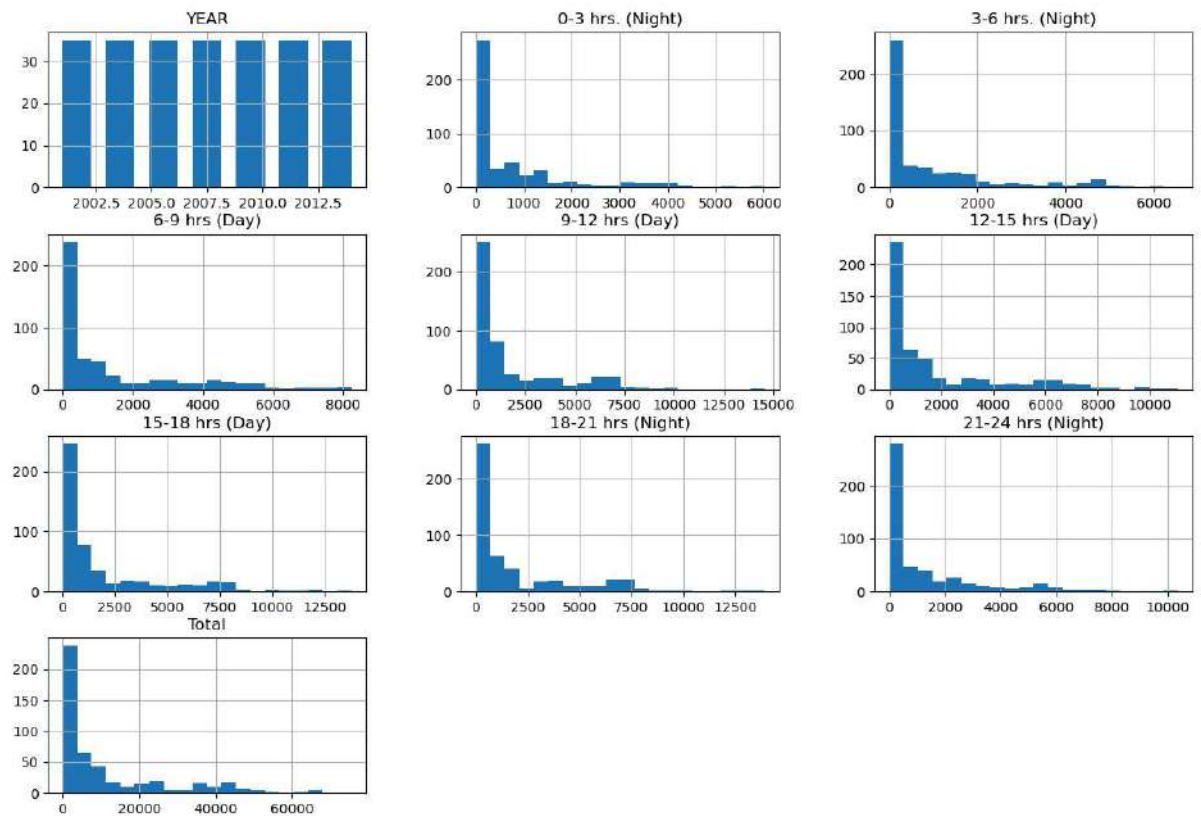
0	7.00
1	1104.25
2	1340.25
3	2037.75
4	2629.25
5	2206.25
6	2230.00
7	2022.00
8	1569.50
9	16105.75

Frequency and Data Distributions using Histogram

```
In [27]: import matplotlib.pyplot as plt
import seaborn as sns

# Load your dataset into a DataFrame
file_path3 = 'only_road_accidents_data3.csv'
data3 = pd.read_csv(file_path3)

# Histograms for all numeric columns
data3.hist(bins=20, figsize=(15, 10))
plt.show()
```



Summary statistics

```
In [51]: # Summary statistics
summary = data3.describe()
print(summary)
```

	YEAR	0-3 hrs. (Night)	3-6 hrs. (Night)	6-9 hrs (Day)	\
count	490.000000	490.000000	490.000000	490.000000	
mean	2007.500000	796.320408	969.236735	1371.15102	
std	4.035249	1210.999326	1415.984501	1864.19157	
min	2001.000000	0.000000	0.000000	0.000000	
25%	2004.000000	10.000000	15.000000	36.000000	
50%	2007.500000	200.000000	251.500000	498.500000	
75%	2011.000000	1114.250000	1355.250000	2073.750000	
max	2014.000000	6020.000000	6545.000000	8241.000000	

	9-12 hrs (Day)	12-15 hrs (Day)	15-18 hrs (Day)	18-21 hrs (Night)	\
count	490.000000	490.000000	490.000000	490.000000	
mean	1753.967347	1681.814286	1850.283673	1782.918367	
std	2400.593532	2326.251665	2643.765262	2587.452453	
min	0.000000	0.000000	0.000000	0.000000	
25%	51.250000	59.250000	59.000000	50.000000	
50%	678.000000	679.500000	669.000000	517.000000	
75%	2680.500000	2265.500000	2289.000000	2072.000000	
max	14568.000000	11089.000000	13789.000000	13927.000000	

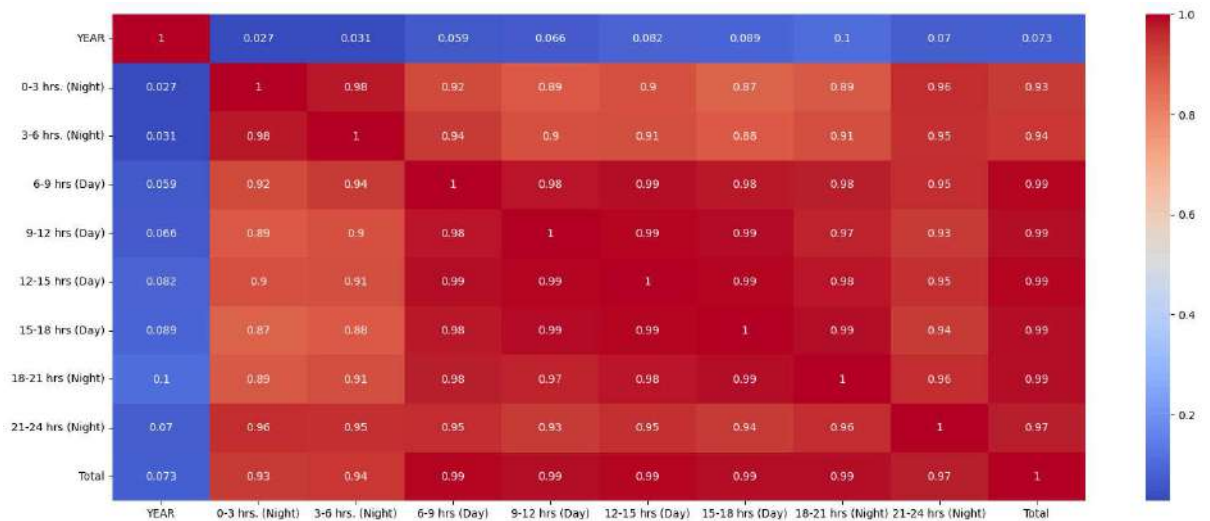
	21-24 hrs (Night)	Total
count	490.000000	490.000000
mean	1228.810204	11434.502041
std	1836.242850	15948.689470
min	0.000000	0.000000
25%	25.250000	357.750000
50%	343.500000	4174.500000
75%	1594.750000	16463.500000
max	10364.000000	75480.000000

Correlation heatmap for numeric columns

```
In [52]: # Correlation heatmap for numeric columns
correlation_matrix = data3.corr()
plt.figure(figsize=(20, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.show()
```

C:\Users\lenovo\AppData\Local\Temp\ipykernel_4388\1515737523.py:2: FutureWarning: The default value of numeric_only in DataFrame.corr is deprecated. In a future version, it will default to False. Select only valid columns or specify the value of numeric_only to silence this warning.

```
correlation_matrix = data3.corr()
```

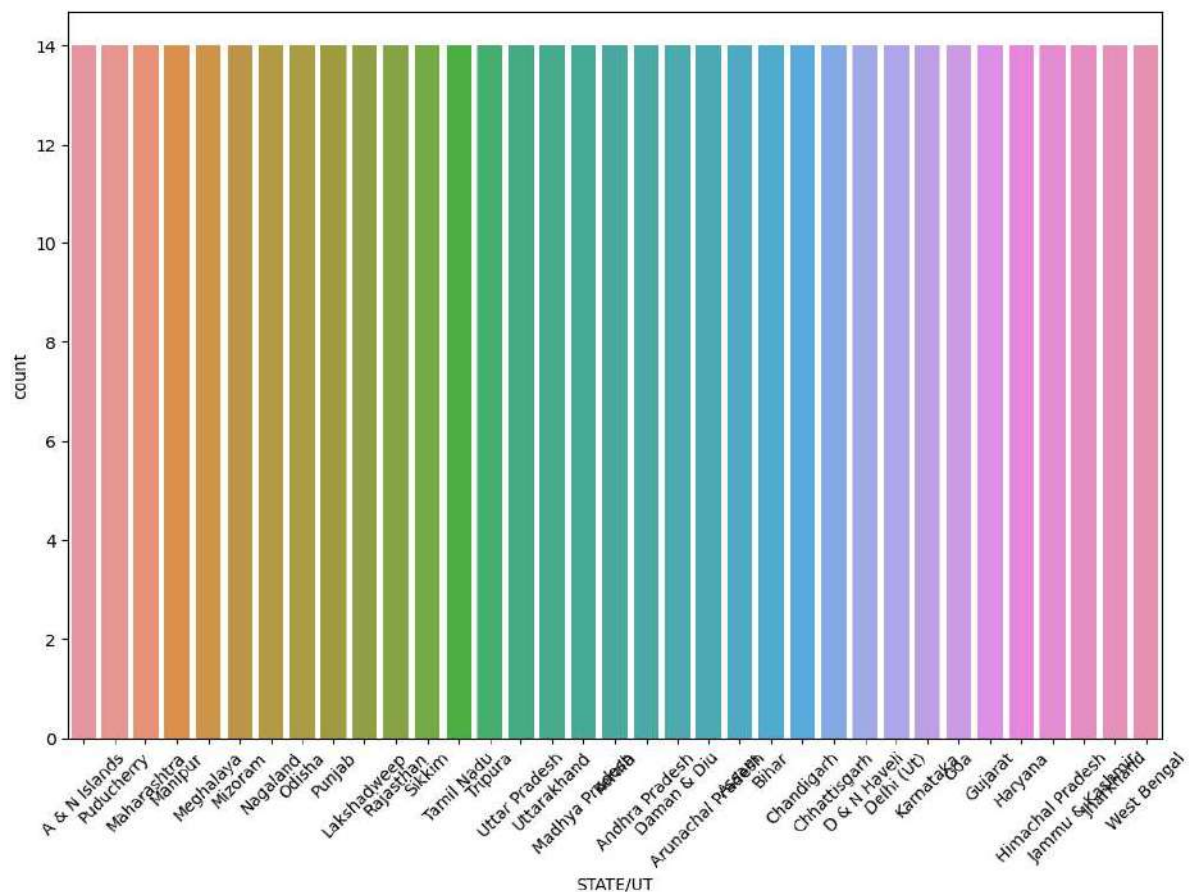


Frequency distribution for categorical columns

```
In [54]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

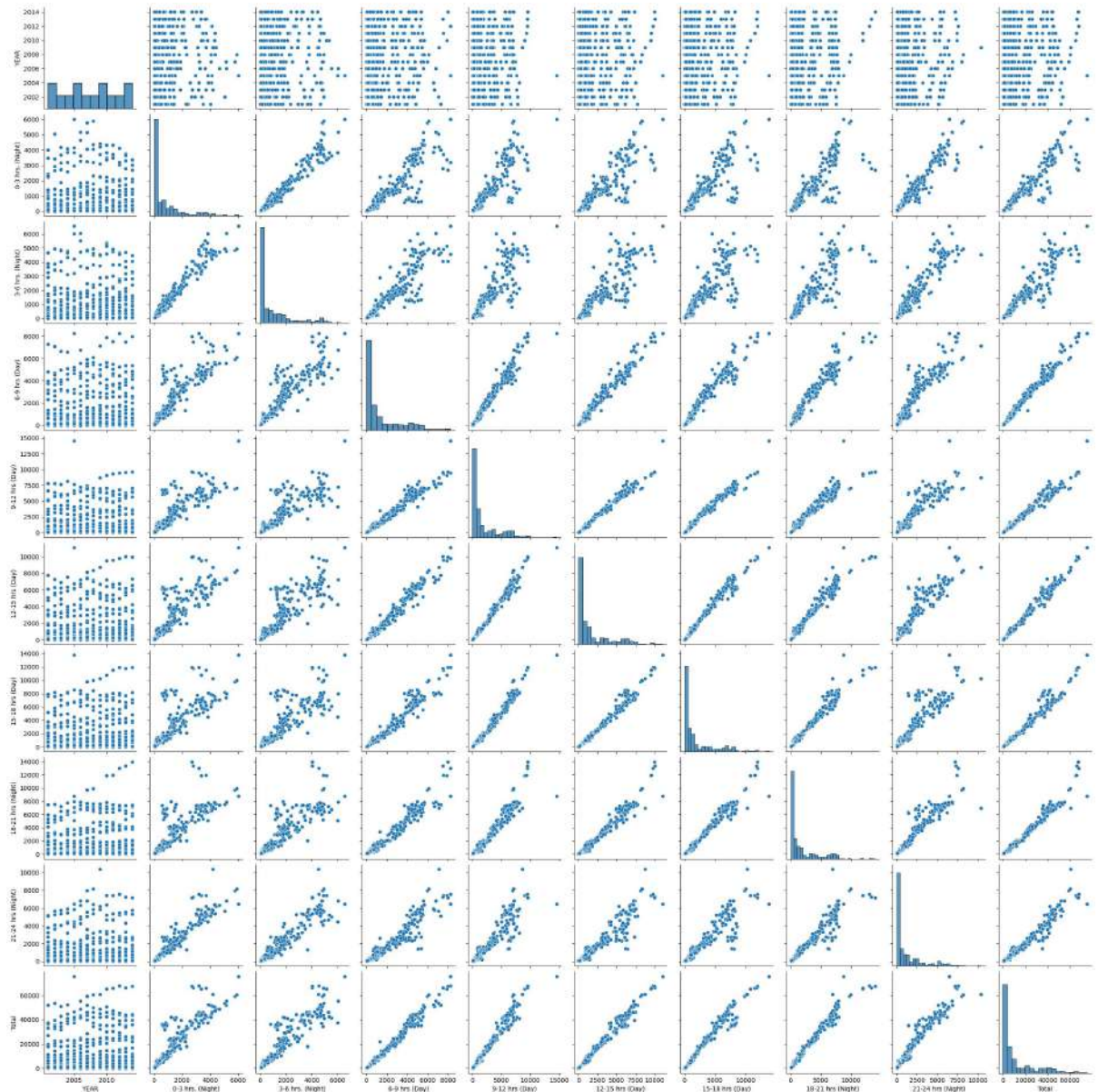
file_path3 = 'only_road_accidents_data3.csv'
data3 = pd.read_csv(file_path3)

# Frequency distribution for categorical columns
categorical_columns = data3.select_dtypes(include='object')
for column in categorical_columns:
    plt.figure(figsize=(12, 8))
    sns.countplot(data=data3, x=column, order=data3[column].value_counts().index)
    plt.xticks(rotation=45)
    plt.show()
```



Pairwise scatter plot matrix for numeric columns

```
In [56]: # Pairwise scatter plot matrix for numeric columns
sns.pairplot(data3, kind='scatter')
plt.show()
```



Find missing values

```
In [59]: # Check for missing values
missing_values = data3.isnull().sum()
print("Missing Values:")
print(missing_values)
```

```
Missing Values:
STATE/UT          0
YEAR              0
0-3 hrs. (Night)  0
3-6 hrs. (Night)  0
6-9 hrs (Day)     0
9-12 hrs (Day)    0
12-15 hrs (Day)   0
15-18 hrs (Day)   0
18-21 hrs (Night) 0
21-24 hrs (Night) 0
Total             0
dtype: int64
```

Check data types

```
In [57]: # Check data types
file_path3 = 'only_road_accidents_data3.csv'
data3 = pd.read_csv(file_path3)

data3_types = data3.dtypes
print("\nData Types:")
print(data3_types)

data3 = pd.read_csv(file_path3)

data3_types = data3.dtypes
print("\nData Types:")
print(data3_types)
```

```
Data Types:
STATE/UT      object
YEAR          int64
0-3 hrs. (Night)  int64
3-6 hrs. (Night)  int64
6-9 hrs (Day)    int64
9-12 hrs (Day)   int64
12-15 hrs (Day)  int64
15-18 hrs (Day)  int64
18-21 hrs (Night) int64
21-24 hrs (Night) int64
Total          int64
dtype: object
```

```
Data Types:
STATE/UT      object
YEAR          int64
0-3 hrs. (Night)  int64
3-6 hrs. (Night)  int64
6-9 hrs (Day)    int64
9-12 hrs (Day)   int64
12-15 hrs (Day)  int64
15-18 hrs (Day)  int64
18-21 hrs (Night) int64
21-24 hrs (Night) int64
Total          int64
dtype: object
```

Check the number of unique values for each column

```
In [58]: unique_values = data3.nunique()  
print("\nUnique Values:")  
print(unique_values)
```

```
Unique Values:  
STATE/UT          35  
YEAR              14  
0-3 hrs. (Night)  316  
3-6 hrs. (Night)  334  
6-9 hrs (Day)     381  
9-12 hrs (Day)    390  
12-15 hrs (Day)   390  
15-18 hrs (Day)   389  
18-21 hrs (Night) 387  
21-24 hrs (Night) 361  
Total            454  
dtype: int64
```

Check for duplicate rows

```
In [60]: duplicate_rows = data3.duplicated().sum()  
print("\nDuplicate Rows:")  
print(duplicate_rows)
```

```
Duplicate Rows:  
0
```

Basic summary statistics

```
In [61]: summary_stats = data3.describe()
print("\nSummary Statistics:")
print(summary_stats)
```

Summary Statistics:

	YEAR	0-3 hrs. (Night)	3-6 hrs. (Night)	6-9 hrs (Day)	\
count	490.000000	490.000000	490.000000	490.000000	
mean	2007.500000	796.320408	969.236735	1371.15102	
std	4.035249	1210.999326	1415.984501	1864.19157	
min	2001.000000	0.000000	0.000000	0.000000	
25%	2004.000000	10.000000	15.000000	36.000000	
50%	2007.500000	200.000000	251.500000	498.50000	
75%	2011.000000	1114.250000	1355.250000	2073.75000	
max	2014.000000	6020.000000	6545.000000	8241.00000	

	9-12 hrs (Day)	12-15 hrs (Day)	15-18 hrs (Day)	18-21 hrs (Night)	\
count	490.000000	490.000000	490.000000	490.000000	
mean	1753.967347	1681.814286	1850.283673	1782.918367	
std	2400.593532	2326.251665	2643.765262	2587.452453	
min	0.000000	0.000000	0.000000	0.000000	
25%	51.250000	59.250000	59.000000	50.000000	
50%	678.000000	679.500000	669.000000	517.000000	
75%	2680.500000	2265.500000	2289.000000	2072.000000	
max	14568.000000	11089.000000	13789.000000	13927.000000	

	21-24 hrs (Night)	Total
count	490.000000	490.000000
mean	1228.810204	11434.502041
std	1836.242850	15948.689470
min	0.000000	0.000000
25%	25.250000	357.750000
50%	343.500000	4174.500000
75%	1594.750000	16463.500000
max	10364.000000	75480.000000

Check for outliers in file

```
In [33]: import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
data3 = pd.read_csv('only_road_accidents_data3.csv')

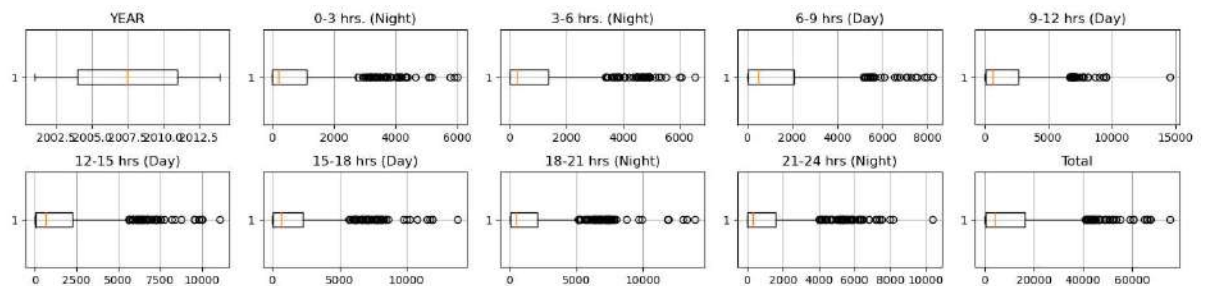
# Select columns with numeric data types (excluding non-numeric columns)
numeric_columns = data3.select_dtypes(include=['int64', 'float64'])

# Set the figure size
plt.figure(figsize=(15, 7))

# Create box plots for each numeric column
for i, column in enumerate(numeric_columns.columns):
    plt.subplot(4, 5, i + 1)
    plt.boxplot(data3[column], vert=False)
    plt.title(column)
    plt.grid(True)

# Adjust the layout
plt.tight_layout()

# Show the box plots
plt.show()
```



Remove Outliers

```
In [82]: import pandas as pd
import numpy as np

def remove_outliers(file_path, threshold=1.5):
    # Read the CSV file into a DataFrame
    data3 = pd.read_csv(file_path)

    # Create a copy of the original data
    original_data = data3.copy()

    # Detect outliers using the IQR method
    Q1 = data3.quantile(0.25, numeric_only=True)
    Q3 = data3.quantile(0.75, numeric_only=True)

    IQR = Q3 - Q1

    lower_bound = Q1 - threshold * IQR
    upper_bound = Q3 + threshold * IQR

    # Align DataFrames and create a boolean mask to identify outliers
    left, right = data3.align(lower_bound, axis=1, copy=False)
    outliers_mask = ((left < right) | (left > right))

    # Remove rows that contain outliers
    data_no_outliers = data3[~outliers_mask.any(axis=1)]

    return original_data, data_no_outliers

# Example usage
file_path = 'only_road_accidents_data3.csv'
original_data, data_no_outliers = remove_outliers(file_path)

# Save the data without outliers to a new CSV file
data_no_outliers.to_csv('data_no_outliers.csv', index=False)
# print(data_no_outliers)
```

Data adequacy assessment summary

```
In [83]: if missing_values.sum() == 0 and duplicate_rows == 0:
    print("\nData appears to be adequate.")
else:
    print("\nData has issues that need to be addressed.")
```

Data appears to be adequate.

Models

linear Regression

```
In [6]: # Import necessary libraries
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Load your dataset (replace 'your_dataset.csv' with the actual dataset file)
data = pd.read_csv('only_road_accidents_data3.csv') # Replace with your actual

# Select the features and target variable
X = data[['0-3 hrs. (Night)', '3-6 hrs. (Night)', '6-9 hrs (Day)', '9-12 hrs (D',
          '15-18 hrs (Day)', '18-21 hrs (Night)', '21-24 hrs (Night)', 'Total']
y = data['Total'] # Replace 'Total' with your actual target variable

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create a Linear Regression model
model = LinearRegression()

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# Print the model coefficients and evaluation metrics
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
print("Mean Squared Error:", mse)
print("R-squared:", r2)
```

```
Coefficients: [0.11111111 0.11111111 0.11111111 0.11111111 0.11111111 0.11111
111
0.11111111 0.11111111 0.88888889]
Intercept: 1.2732925824820995e-11
Mean Squared Error: 5.221461714685703e-22
R-squared: 1.0
```

Random Forest

```
In [14]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load your dataset (replace 'your_dataset.csv' with the actual dataset file)
data = pd.read_csv('only_road_accidents_data3.csv')

# Select the features and target variable
X = data[['0-3 hrs. (Night)', '3-6 hrs. (Night)', '6-9 hrs (Day)', '9-12 hrs (D',
          '15-18 hrs (Day)', '18-21 hrs (Night)', '21-24 hrs (Night)', 'Total']
y = data['STATE/UT'] # Change the target variable to match your dataset

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create a Random Forest classifier
model = RandomForestClassifier(n_estimators=100, random_state=42) # You can ad

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

# Print the model's accuracy and classification report
print("Accuracy:", accuracy)
print("Classification Report:\n", report)
```

Accuracy: 0.8163265306122449

Classification Report:

	precision	recall	f1-score	support
A & N Islands	1.00	1.00	1.00	3
Andhra Pradesh	1.00	1.00	1.00	3
Arunachal Pradesh	1.00	1.00	1.00	3
Assam	1.00	1.00	1.00	1
Bihar	1.00	0.67	0.80	3
Chandigarh	1.00	1.00	1.00	9
Chhattisgarh	0.00	0.00	0.00	4
D & N Haveli	1.00	1.00	1.00	2
Daman & Diu	1.00	1.00	1.00	2
Delhi (Ut)	1.00	1.00	1.00	2
Goa	0.67	1.00	0.80	2
Gujarat	0.50	1.00	0.67	2
Haryana	0.67	1.00	0.80	2
Himachal Pradesh	1.00	1.00	1.00	2
Jammu & Kashmir	0.67	1.00	0.80	2
Jharkhand	0.67	1.00	0.80	2
Karnataka	1.00	1.00	1.00	2
Lakshadweep	1.00	1.00	1.00	2
Madhya Pradesh	1.00	0.67	0.80	3
Maharashtra	1.00	1.00	1.00	1
Manipur	0.40	1.00	0.57	2
Meghalaya	1.00	0.83	0.91	6
Mizoram	0.60	1.00	0.75	3
Nagaland	0.00	0.00	0.00	2
Odisha	0.33	1.00	0.50	1
Puducherry	0.50	0.50	0.50	2
Punjab	1.00	1.00	1.00	3
Rajasthan	1.00	0.75	0.86	4
Sikkim	0.00	0.00	0.00	1
Tamil Nadu	1.00	1.00	1.00	9
Tripura	1.00	0.50	0.67	2
Uttar Pradesh	0.00	0.00	0.00	1
Uttarakhand	0.50	0.20	0.29	5
West Bengal	0.83	1.00	0.91	5
accuracy			0.82	98
macro avg	0.75	0.80	0.75	98
weighted avg	0.81	0.82	0.79	98

```
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and bei
ng set to 0.0 in labels with no predicted samples. Use `zero_division` parame
ter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and bei
ng set to 0.0 in labels with no predicted samples. Use `zero_division` parame
ter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and bei
ng set to 0.0 in labels with no predicted samples. Use `zero_division` parame
ter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
```

SVM Model

```
In [19]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# Create a sample dataset (replace this with your actual dataset)
data = {
    '0-3 hrs. (Night)': [1, 2, 3, 4, 5],
    '3-6 hrs. (Night)': [0, 2, 1, 4, 3],
    '6-9 hrs (Day)': [5, 3, 4, 2, 1],
    '9-12 hrs (Day)': [4, 1, 5, 3, 2],
    '12-15 hrs (Day)': [2, 4, 1, 5, 3],
    '15-18 hrs (Day)': [3, 5, 2, 1, 4],
    '18-21 hrs (Night)': [1, 3, 2, 5, 4],
    '21-24 hrs (Night)': [4, 1, 5, 2, 3],
    'Total': [10, 11, 12, 13, 14],
    'Casualty Severity': [0, 1, 0, 1, 0] # Sample target variable (0 or 1)
}

# Create a DataFrame from the sample data
df = pd.DataFrame(data)

# Select the features and target variable
X = df[['0-3 hrs. (Night)', '3-6 hrs. (Night)', '6-9 hrs (Day)', '9-12 hrs (Day)',
        '12-15 hrs (Day)', '15-18 hrs (Day)', '18-21 hrs (Night)', '21-24 hrs (Night)']]
y = df['Casualty Severity']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create an SVM classifier with a linear kernel
model = SVC(kernel='linear', random_state=42)

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

# Print the model's accuracy and classification report
print("Accuracy:", accuracy)
print("Classification Report:\n", report)
```

Accuracy: 0.0

Classification Report:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	0.0
1	0.00	0.00	0.00	1.0
accuracy			0.00	1.0
macro avg	0.00	0.00	0.00	1.0
weighted avg	0.00	0.00	0.00	1.0

C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Recall and F-score are ill-defined and being set to 0.0 in labels with no true samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Recall and F-score are ill-defined and being set to 0.0 in labels with no true samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Recall and F-score are ill-defined and being set to 0.0 in labels with no true samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))

Decision Trees

```
In [5]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load your dataset (replace 'your_dataset.csv' with the actual dataset file)
data = pd.read_csv('only_road_accidents_data3.csv') # Replace with your actual

# Select the features and target variable
X = data[['0-3 hrs. (Night)', '3-6 hrs. (Night)', '6-9 hrs (Day)', '9-12 hrs (D',
          '15-18 hrs (Day)', '18-21 hrs (Night)', '21-24 hrs (Night)', 'Total']
y = data['Total'] # Replace 'Total' with your actual target variable

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create a Decision Tree classifier
model = DecisionTreeClassifier(random_state=42) # You can set various hyperpar

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

# Print the model's accuracy and classification report
print("Accuracy:", accuracy)
print("Classification Report:\n", report)
```

Accuracy: 0.04081632653061224

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	1
2	0.00	0.00	0.00	0
30	0.00	0.00	0.00	1
45	0.00	0.00	0.00	0
50	0.00	0.00	0.00	0
53	0.00	0.00	0.00	1
55	0.00	0.00	0.00	0
56	0.00	0.00	0.00	1
57	0.00	0.00	0.00	1
72	1.00	1.00	1.00	1
75	0.00	0.00	0.00	0
87	1.00	1.00	1.00	1
95	0.00	0.00	0.00	1
103	0.00	0.00	0.00	0
111	0.00	0.00	0.00	1
116	0.00	0.00	0.00	1
125	0.00	0.00	0.00	0
150	0.00	0.00	0.00	0
157	0.00	0.00	0.00	1
181	0.00	0.00	0.00	1
196	0.00	0.00	0.00	0
204	0.00	0.00	0.00	1
217	0.00	0.00	0.00	0
229	0.00	0.00	0.00	1
235	0.00	0.00	0.00	0
236	0.00	0.00	0.00	1
237	0.00	0.00	0.00	0
243	0.00	0.00	0.00	1
257	0.00	0.00	0.00	1
261	0.00	0.00	0.00	0
271	0.00	0.00	0.00	0
280	0.00	0.00	0.00	0
285	0.00	0.00	0.00	1
314	0.00	0.00	0.00	1
328	0.00	0.00	0.00	1
330	0.00	0.00	0.00	1
351	0.00	0.00	0.00	1
366	0.00	0.00	0.00	0
373	0.00	0.00	0.00	1
409	0.00	0.00	0.00	1
411	0.00	0.00	0.00	1
412	0.00	0.00	0.00	0
424	0.00	0.00	0.00	1
441	0.00	0.00	0.00	0
443	0.00	0.00	0.00	1
456	0.00	0.00	0.00	1
477	0.00	0.00	0.00	1
493	0.00	0.00	0.00	1
520	0.00	0.00	0.00	0
521	0.00	0.00	0.00	1
536	0.00	0.00	0.00	1
544	0.00	0.00	0.00	1

600	0.00	0.00	0.00	1
671	0.00	0.00	0.00	0
691	0.00	0.00	0.00	1
767	0.00	0.00	0.00	0
771	0.00	0.00	0.00	0
801	0.00	0.00	0.00	1
865	0.00	0.00	0.00	1
880	0.00	0.00	0.00	1
995	0.00	0.00	0.00	0
1003	0.00	0.00	0.00	1
1117	0.00	0.00	0.00	0
1286	0.00	0.00	0.00	1
1383	0.00	0.00	0.00	0
1409	0.00	0.00	0.00	1
1451	0.00	0.00	0.00	1
1455	0.00	0.00	0.00	1
1461	0.00	0.00	0.00	1
1510	0.00	0.00	0.00	0
1522	0.00	0.00	0.00	0
1586	0.00	0.00	0.00	0
1806	0.00	0.00	0.00	0
1808	0.00	0.00	0.00	1
2036	0.00	0.00	0.00	1
2110	0.00	0.00	0.00	0
2899	0.00	0.00	0.00	1
2981	0.00	0.00	0.00	0
3325	0.00	0.00	0.00	1
3901	0.00	0.00	0.00	1
3902	0.00	0.00	0.00	0
4022	0.00	0.00	0.00	0
4170	0.00	0.00	0.00	1
4230	0.00	0.00	0.00	1
4334	0.00	0.00	0.00	1
4382	0.00	0.00	0.00	0
4574	0.00	0.00	0.00	0
4987	0.00	0.00	0.00	1
5631	0.00	0.00	0.00	0
5669	0.00	0.00	0.00	0
5778	0.00	0.00	0.00	1
6012	0.00	0.00	0.00	1
6142	0.00	0.00	0.00	0
6180	0.00	0.00	0.00	1
6323	0.00	0.00	0.00	1
6424	0.00	0.00	0.00	0
6496	0.00	0.00	0.00	0
6535	0.00	0.00	0.00	0
6637	0.00	0.00	0.00	0
6675	0.00	0.00	0.00	0
7023	0.00	0.00	0.00	1
7144	0.00	0.00	0.00	1
7614	0.00	0.00	0.00	0
7802	0.00	0.00	0.00	1
7837	0.00	0.00	0.00	1
8184	0.00	0.00	0.00	0
8366	0.00	0.00	0.00	0
9282	0.00	0.00	0.00	1
9285	0.00	0.00	0.00	0

9398	0.00	0.00	0.00	1
9413	0.00	0.00	0.00	0
9699	0.00	0.00	0.00	1
10198	0.00	0.00	0.00	1
10262	0.00	0.00	0.00	1
10473	0.00	0.00	0.00	0
11043	0.00	0.00	0.00	1
11161	0.00	0.00	0.00	0
11237	0.00	0.00	0.00	0
11375	0.00	0.00	0.00	0
11430	0.00	0.00	0.00	0
11547	0.00	0.00	0.00	1
12181	0.00	0.00	0.00	1
13085	0.00	0.00	0.00	1
13511	0.00	0.00	0.00	0
13959	0.00	0.00	0.00	1
14689	0.00	0.00	0.00	1
14725	0.00	0.00	0.00	1
15608	0.00	0.00	0.00	0
18478	0.00	0.00	0.00	0
18740	0.00	0.00	0.00	1
19808	0.00	0.00	0.00	1
20101	0.00	0.00	0.00	0
20571	0.00	0.00	0.00	1
21933	0.00	0.00	0.00	0
22969	0.00	0.00	0.00	1
23348	0.00	0.00	0.00	1
23586	0.00	0.00	0.00	0
23591	0.00	0.00	0.00	0
23704	0.00	0.00	0.00	0
23885	0.00	0.00	0.00	1
25038	0.00	0.00	0.00	1
25975	0.00	0.00	0.00	0
28444	0.00	0.00	0.00	1
30127	0.00	0.00	0.00	1
30199	0.00	0.00	0.00	0
32577	0.00	0.00	0.00	1
34537	0.00	0.00	0.00	0
37269	0.00	0.00	0.00	0
37289	0.00	0.00	0.00	1
41323	0.00	0.00	0.00	1
43280	0.00	0.00	0.00	1
45247	0.00	0.00	0.00	0
46252	0.00	0.00	0.00	1
46334	0.00	0.00	0.00	0
46586	0.00	0.00	0.00	0
47120	0.00	0.00	0.00	1
50619	0.00	0.00	0.00	0
51025	0.00	0.00	0.00	1
51975	0.00	0.00	0.00	0
51978	0.00	0.00	0.00	1
53503	0.00	0.00	0.00	0
59117	0.00	0.00	0.00	1
60409	0.00	0.00	0.00	1
60794	0.00	0.00	0.00	1
64996	0.00	0.00	0.00	1
65873	0.00	0.00	0.00	1

67757	0.00	0.00	0.00	1
75480	0.00	0.00	0.00	1
accuracy			0.04	98
macro avg	0.02	0.02	0.02	98
weighted avg	0.04	0.04	0.04	98

C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))

C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Recall and F-score are ill-defined and being set to 0.0 in labels with no true samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))

C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))

C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Recall and F-score are ill-defined and being set to 0.0 in labels with no true samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))

C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))

C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Recall and F-score are ill-defined and being set to 0.0 in labels with no true samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))

Evaluation Metrics for DataSet 3

Accuracy Metrics

```
In [18]: # Import necessary Libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load your dataset
data = pd.read_csv('only_road_accidents_data3.csv')

# Select the features and target variable
X = data[['0-3 hrs. (Night)', '3-6 hrs. (Night)', '6-9 hrs (Day)', '9-12 hrs (Day)']]
y = data['Total'] # Replace 'Total' with the correct target variable from your dataset

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create a Decision Tree classifier
model = DecisionTreeClassifier(random_state=42)

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

# Print the accuracy
print("Accuracy:", accuracy)
```

Accuracy: 0.04081632653061224

Precision and Recall Metrics

```
In [19]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import precision_score, recall_score

# Load your dataset
data = pd.read_csv('only_road_accidents_data3.csv')

# Select the features and target variable
X = data[['0-3 hrs. (Night)', '3-6 hrs. (Night)', '6-9 hrs (Day)', '9-12 hrs (Day)']]
y = data['Total'] # Replace 'Total' with the correct target variable from your dataset

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create a Decision Tree classifier
model = DecisionTreeClassifier(random_state=42)

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Calculate Precision and Recall (weighted average)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')

# Print the Precision and Recall
print("Precision:", precision)
print("Recall:", recall)
```

Precision: 0.04081632653061224

Recall: 0.04081632653061224

C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))

C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in labels with no true samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))

Mean Absolute Error (MAE) or Mean Squared Error (MSE) Metrics

```
In [20]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error

# Load your dataset
data = pd.read_csv('only_road_accidents_data3.csv')

# Select the features and target variable
X = data[['0-3 hrs. (Night)', '3-6 hrs. (Night)', '6-9 hrs (Day)', '9-12 hrs (Day)']]
y = data['Total'] # Replace 'Total' with the correct target variable from your dataset

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create a Decision Tree Regressor (for regression)
model = DecisionTreeRegressor(random_state=42)

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Calculate Mean Absolute Error (MAE)
mae = mean_absolute_error(y_test, y_pred)

# Calculate Mean Squared Error (MSE)
mse = mean_squared_error(y_test, y_pred)

# Print MAE and MSE
print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
```

Mean Absolute Error (MAE): 431.8979591836735
Mean Squared Error (MSE): 2019834.5918367347

Log Loss (Logarithmic Loss) Metrics

```
In [ ]: the dataset is continuous numeric values (e.g., time intervals), Log Loss is not used.
Log Loss is used when you have a classification problem with multiple classes,
```

DataSet 4

```
In [84]: file_path4 = 'only_road_accidents_data_month2.csv'
data4 = pd.read_csv(file_path4)
```

```
In [85]: # Assuming 'data4' is your DataFrame
null_counts4 = data4.isnull().sum()

# Display the null value counts
# print(null_counts4)

df4 = pd.DataFrame(null_counts4)

# Check for missing data
missing_data4 = df4.isna()

# Count missing values in each column
missing_count4 = df4.isna().sum()

# Print the DataFrame with missing data
print("Original DataFrame:")
print(df4)
```

Original DataFrame:

	0
STATE/UT	0
YEAR	0
JANUARY	0
FEBRUARY	0
MARCH	0
APRIL	0
MAY	0
JUNE	0
JULY	0
AUGUST	0
SEPTEMBER	0
OCTOBER	0
NOVEMBER	0
DECEMBER	0
TOTAL	0

Missing Data

```
In [87]: # Print missing data status
print("\nMissing Data Status:")
print(missing_data4)
```

Missing Data Status:

```
0
STATE/UT  False
YEAR      False
JANUARY   False
FEBRUARY  False
MARCH     False
APRIL     False
MAY       False
JUNE      False
JULY      False
AUGUST    False
SEPTEMBER False
OCTOBER   False
NOVEMBER  False
DECEMBER  False
TOTAL     False
```

Print missing data counts

```
In [88]: print("\nMissing Data Counts:")
print(missing_count4)
```

Missing Data Counts:

```
0 0
dtype: int64
```

Drop Rows with Missing Values

```
In [89]: df4_dropped4 = df4.dropna()
```

Fill Missing Values with a Specific Value

```
In [90]: df4_filled4 = df4.fillna(0)
```

Print DataFrames after handling missing data

```
In [92]: print("\nDataFrame after dropping rows with missing data:")  
print(df4_dropped4)
```

DataFrame after dropping rows with missing data:

	0
STATE/UT	0
YEAR	0
JANUARY	0
FEBRUARY	0
MARCH	0
APRIL	0
MAY	0
JUNE	0
JULY	0
AUGUST	0
SEPTEMBER	0
OCTOBER	0
NOVEMBER	0
DECEMBER	0
TOTAL	0

```
In [93]: # print("\nDataFrame after filling missing data with 0:")  
print(df4_filled4)
```

	0
STATE/UT	0
YEAR	0
JANUARY	0
FEBRUARY	0
MARCH	0
APRIL	0
MAY	0
JUNE	0
JULY	0
AUGUST	0
SEPTEMBER	0
OCTOBER	0
NOVEMBER	0
DECEMBER	0
TOTAL	0

```
In [98]: from sklearn.preprocessing import MinMaxScaler

# Initialize the MinMaxScaler
scaler = MinMaxScaler()

# Fit and transform the DataFrame to normalize the data
normalized_data4 = scaler.fit_transform(df4)

# Create a new DataFrame with the normalized values
normalized_df4 = pd.DataFrame(normalized_data4, columns=df4.columns)
```

Original DataFrame

```
In [100]: print("Original DataFrame:")
print(df4)
```

Original DataFrame:

	0
STATE/UT	0
YEAR	0
JANUARY	0
FEBRUARY	0
MARCH	0
APRIL	0
MAY	0
JUNE	0
JULY	0
AUGUST	0
SEPTEMBER	0
OCTOBER	0
NOVEMBER	0
DECEMBER	0
TOTAL	0

Normalized DataFrame

```
In [102]: # print("\nNormalized DataFrame:")  
print(normalized_df4)
```

```
      0  
0  0.0  
1  0.0  
2  0.0  
3  0.0  
4  0.0  
5  0.0  
6  0.0  
7  0.0  
8  0.0  
9  0.0  
10 0.0  
11 0.0  
12 0.0  
13 0.0  
14 0.0
```

Check for Duplicate Columns and Rows

```
In [103]: import pandas as pd

# Load your data from the CSV file
data4 = pd.read_csv('only_road_accidents_data_month2.csv')

# Identify duplicate columns
duplicate_columns = []
seen_columns = set()
for column in data4.columns:
    if column in seen_columns:
        duplicate_columns.append(column)
    else:
        seen_columns.add(column)

# Identify duplicate rows
duplicate_rows = data4[data4.duplicated()]

if duplicate_columns:
    print("Duplicate columns found:")
    for column in duplicate_columns:
        print(column)
else:
    print("No duplicate columns found.")

if not duplicate_rows.empty:
    print("\nDuplicate rows found:")
    print(duplicate_rows)
else:
    print("No duplicate rows found.")
```

No duplicate columns found.

No duplicate rows found.

Check whether the dataset is imbalanced.

```
In [106]: import pandas as pd

file_path4 = 'only_road_accidents_data_month2.csv'
data4 = pd.read_csv(file_path4)

df4 = pd.DataFrame(data4)

# Identify columns with imbalanced data (categorical columns with significantly
imbalanced_columns = []

for column in df4.columns:
    if df4[column].dtype == 'object':
        value_counts = df4[column].value_counts()
        if len(value_counts) <= 1 or value_counts.max() / value_counts.min() >= 10:
            imbalanced_columns.append(column)

print("Imbalanced Columns:")
print(imbalanced_columns)

Imbalanced Columns:
['STATE/UT']
```

Generate Descriptive Analysis

```
In [107]: import matplotlib.pyplot as plt
import seaborn as sns

# Load your dataset into a DataFrame
file_path4 = 'only_road_accidents_data_month2.csv'
data4 = pd.read_csv(file_path4)
```

Mean , Median or Mode

```
In [108]: import pandas as pd
import numpy as np
from statistics import mode

# Load your data from the CSV file
data4 = pd.read_csv('only_road_accidents_data_month2.csv')

# Filter only the numeric columns
numeric_columns = data4.select_dtypes(include=[np.number])

# Calculate and display the mean, median, and mode for each numeric column
for column in numeric_columns.columns:
    mean = np.mean(numeric_columns[column])
    median = np.median(numeric_columns[column])

    # Flatten the column into a 1D array for mode calculation
    flat_data4 = numeric_columns[column].values.flatten()
    try:
        mode_value = mode(flat_data4)
    except StatisticsError:
        mode_value = "No unique mode found"

    print(f"Column: {column}")
    print(f"Mean: {mean}")
    print(f"Median: {median}")
    print(f"Mode: {mode_value}")
    print()
```

Column: YEAR
Mean: 2007.5
Median: 2007.5
Mode: 2001

Column: JANUARY
Mean: 985.1408163265306
Median: 329.5
Mode: 6

Column: FEBRUARY
Mean: 937.2897959183673
Median: 348.5
Mode: 0

Column: MARCH
Mean: 992.1244897959184
Median: 369.0
Mode: 0

Column: APRIL
Mean: 978.9040816326531
Median: 350.0
Mode: 0

Column: MAY
Mean: 1064.4142857142858
Median: 387.5
Mode: 0

Column: JUNE
Mean: 966.4775510204082
Median: 342.5
Mode: 0

Column: JULY
Mean: 898.4959183673469
Median: 308.0
Mode: 0

Column: AUGUST
Mean: 894.5938775510205
Median: 313.0
Mode: 0

Column: SEPTEMBER
Mean: 888.3714285714286
Median: 311.5
Mode: 0

Column: OCTOBER
Mean: 928.4918367346938
Median: 311.5
Mode: 0

Column: NOVEMBER
Mean: 933.0448979591837

Median: 331.0

Mode: 0

Column: DECEMBER

Mean: 967.1530612244898

Median: 347.5

Mode: 0

Column: TOTAL

Mean: 11434.502040816327

Median: 4174.5

Mode: 0

Range, Variance, Standard Deviation or Interquartile Range (IQR)

```
In [109]: import pandas as pd
import numpy as np

# Load your data from the CSV file
data4 = pd.read_csv('only_road_accidents_data_month2.csv')

# Select only numeric columns
numeric_data = data4.select_dtypes(include=[np.number])

# Calculate measures of dispersion for all numeric columns
dispersion_metrics = {
    'Column': [],
    'Range': [],
    'Variance': [],
    'Standard Deviation': [],
    'Interquartile Range (IQR)': []
}

for column in numeric_data.columns:
    dispersion_metrics['Column'].append(column)
    dispersion_metrics['Range'].append(np.ptp(numeric_data[column]))
    dispersion_metrics['Variance'].append(np.var(numeric_data[column]))
    dispersion_metrics['Standard Deviation'].append(np.std(numeric_data[column]))
    q1 = numeric_data[column].quantile(0.25)
    q3 = numeric_data[column].quantile(0.75)
    dispersion_metrics['Interquartile Range (IQR)'].append(q3 - q1)

# Create a summary DataFrame
dispersion_df = pd.DataFrame(dispersion_metrics)

# Print the results
print(dispersion_df)
```

	Column	Range	Variance	Standard Deviation \
0	YEAR	13	1.625000e+01	4.031129
1	JANUARY	9062	1.967085e+06	1402.528122
2	FEBRUARY	8705	1.757147e+06	1325.574270
3	MARCH	9205	1.965638e+06	1402.012298
4	APRIL	9006	1.948662e+06	1395.944897
5	MAY	9330	2.269923e+06	1506.626498
6	JUNE	5915	1.805514e+06	1343.694312
7	JULY	5719	1.548865e+06	1244.533968
8	AUGUST	5936	1.593598e+06	1262.377886
9	SEPTEMBER	5777	1.560759e+06	1249.303230
10	OCTOBER	5516	1.620339e+06	1272.925256
11	NOVEMBER	5367	1.608068e+06	1268.096022
12	DECEMBER	5562	1.786238e+06	1336.502242
13	TOTAL	75480	2.538416e+08	15932.406986

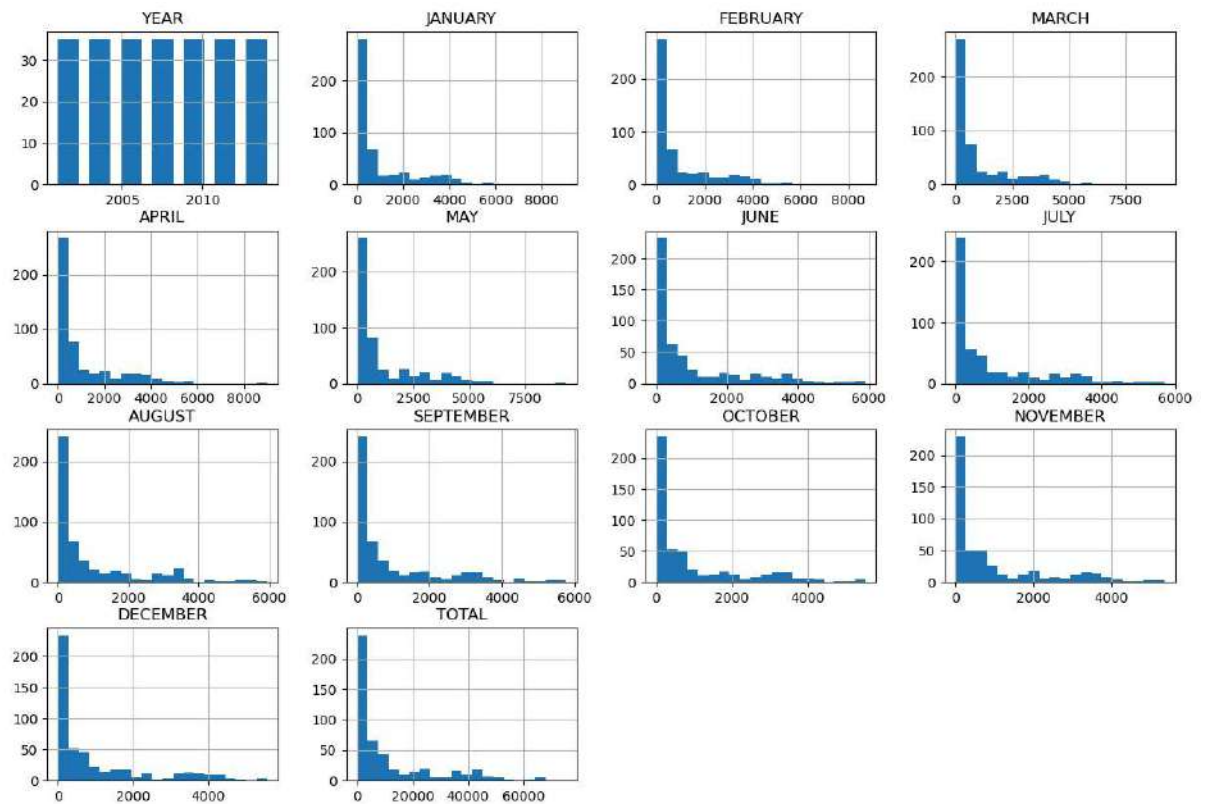
	Interquartile Range (IQR)
0	7.00
1	1482.50
2	1364.25
3	1348.75
4	1331.00
5	1431.75
6	1356.00
7	1273.25
8	1193.75
9	1199.75
10	1256.75
11	1218.25
12	1335.25
13	16105.75

Frequency and Data Distributions using Histogram

```
In [29]: import matplotlib.pyplot as plt
import seaborn as sns

# Load your dataset into a DataFrame
file_path4 = 'only_road_accidents_data_month2.csv'
data4 = pd.read_csv(file_path4)

# Histograms for all numeric columns
data4.hist(bins=20, figsize=(15, 10))
plt.show()
```



Summary statistics

```
In [111]: # Summary statistics
summary = data4.describe()
print(summary)
```

	YEAR	JANUARY	FEBRUARY	MARCH	APRIL	\
count	490.000000	490.000000	490.000000	490.000000	490.000000	
mean	2007.500000	985.140816	937.289796	992.124490	978.904082	
std	4.035249	1403.961467	1326.928970	1403.445117	1397.371515	
min	2001.000000	0.000000	0.000000	0.000000	0.000000	
25%	2004.000000	31.000000	30.000000	32.250000	25.250000	
50%	2007.500000	329.500000	348.500000	369.000000	350.000000	
75%	2011.000000	1513.500000	1394.250000	1381.000000	1356.250000	
max	2014.000000	9062.000000	8705.000000	9205.000000	9006.000000	

	MAY	JUNE	JULY	AUGUST	SEPTEMBER	\
count	490.000000	490.000000	490.000000	490.000000	490.000000	
mean	1064.414286	966.477551	898.495918	894.593878	888.371429	
std	1508.166230	1345.067531	1245.805847	1263.668002	1250.579984	
min	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	32.250000	28.000000	28.250000	28.500000	27.750000	
50%	387.500000	342.500000	308.000000	313.000000	311.500000	
75%	1464.000000	1384.000000	1301.500000	1222.250000	1227.500000	
max	9330.000000	5915.000000	5719.000000	5936.000000	5777.000000	

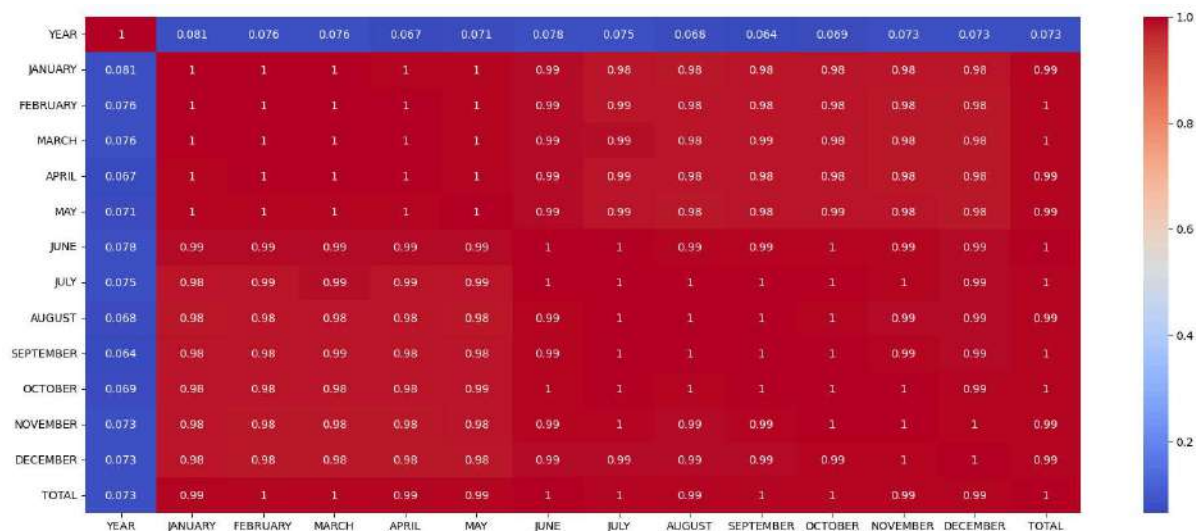
	OCTOBER	NOVEMBER	DECEMBER	TOTAL
count	490.000000	490.000000	490.000000	490.000000
mean	928.491837	933.044898	967.153061	11434.502041
std	1274.226151	1269.391981	1337.868111	15948.689470
min	0.000000	0.000000	0.000000	0.000000
25%	30.250000	28.250000	30.000000	357.750000
50%	311.500000	331.000000	347.500000	4174.500000
75%	1287.000000	1246.500000	1365.250000	16463.500000
max	5516.000000	5367.000000	5562.000000	75480.000000

Correlation heatmap for numeric columns

```
In [113]: # Correlation heatmap for numeric columns
correlation_matrix = data4.corr()
plt.figure(figsize=(20, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.show()
```

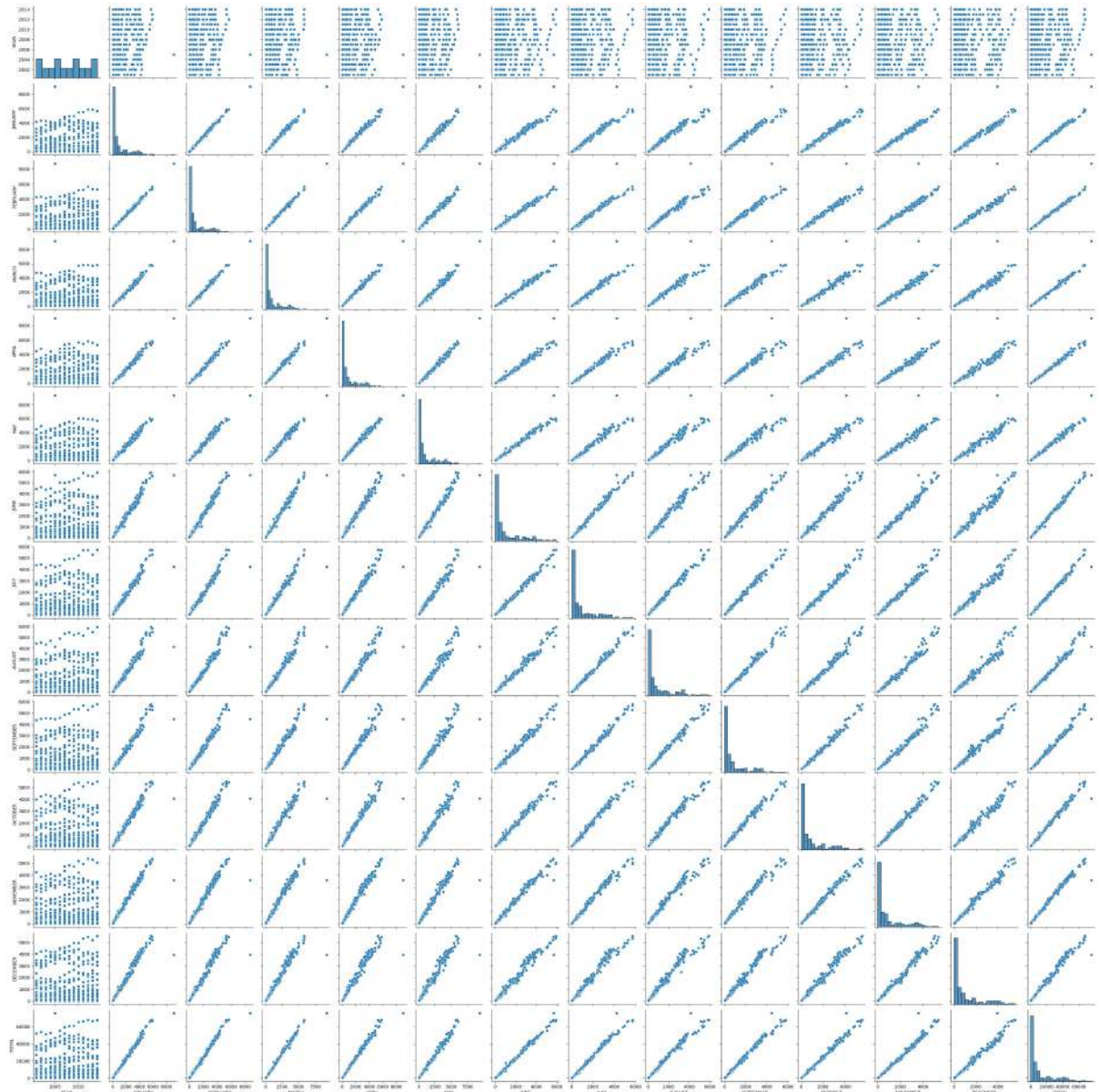
C:\Users\lenovo\AppData\Local\Temp\ipykernel_4388\2707473026.py:2: FutureWarning: The default value of numeric_only in DataFrame.corr is deprecated. In a future version, it will default to False. Select only valid columns or specify the value of numeric_only to silence this warning.

```
correlation_matrix = data4.corr()
```



Pairwise scatter plot matrix for numeric columns

```
In [115]: # Pairwise scatter plot matrix for numeric columns
sns.pairplot(data4, kind='scatter')
plt.show()
```



Find missing values

```
In [120]: # Check for missing values
missing_values = data4.isnull().sum()
print("Missing Values:")
print(missing_values)
```

Missing Values:

STATE/UT	0
YEAR	0
JANUARY	0
FEBRUARY	0
MARCH	0
APRIL	0
MAY	0
JUNE	0
JULY	0
AUGUST	0
SEPTEMBER	0
OCTOBER	0
NOVEMBER	0
DECEMBER	0
TOTAL	0

dtype: int64

Check data types

```
In [119]: # Check data types
file_path4 = 'only_road_accidents_data_month2.csv'
data4 = pd.read_csv(file_path4)

data4_types = data4.dtypes
print("\nData Types:")
print(data4_types)
```

```
Data Types:
STATE/UT      object
YEAR          int64
JANUARY       int64
FEBRUARY      int64
MARCH         int64
APRIL         int64
MAY          int64
JUNE         int64
JULY         int64
AUGUST       int64
SEPTEMBER    int64
OCTOBER      int64
NOVEMBER     int64
DECEMBER     int64
TOTAL        int64
dtype: object
```

Check the number of unique values for each column

```
In [117]: unique_values = data4.nunique()  
print("\nUnique Values:")  
print(unique_values)
```

```
Unique Values:  
STATE/UT      37  
YEAR          14  
JANUARY      357  
FEBRUARY     356  
MARCH        349  
APRIL        349  
MAY          359  
JUNE         358  
JULY         355  
AUGUST       353  
SEPTEMBER    345  
OCTOBER      358  
NOVEMBER     350  
DECEMBER     350  
TOTAL        454  
dtype: int64
```

Check for duplicate rows

```
In [116]: duplicate_rows = data4.duplicated().sum()  
print("\nDuplicate Rows:")  
print(duplicate_rows)
```

```
Duplicate Rows:  
0
```

Basic summary statistics

```
In [121]: summary_stats = data4.describe()
print("\nSummary Statistics:")
print(summary_stats)
```

Summary Statistics:

	YEAR	JANUARY	FEBRUARY	MARCH	APRIL \
count	490.000000	490.000000	490.000000	490.000000	490.000000
mean	2007.500000	985.140816	937.289796	992.124490	978.904082
std	4.035249	1403.961467	1326.928970	1403.445117	1397.371515
min	2001.000000	0.000000	0.000000	0.000000	0.000000
25%	2004.000000	31.000000	30.000000	32.250000	25.250000
50%	2007.500000	329.500000	348.500000	369.000000	350.000000
75%	2011.000000	1513.500000	1394.250000	1381.000000	1356.250000
max	2014.000000	9062.000000	8705.000000	9205.000000	9006.000000

	MAY	JUNE	JULY	AUGUST	SEPTEMBER \
count	490.000000	490.000000	490.000000	490.000000	490.000000
mean	1064.414286	966.477551	898.495918	894.593878	888.371429
std	1508.166230	1345.067531	1245.805847	1263.668002	1250.579984
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	32.250000	28.000000	28.250000	28.500000	27.750000
50%	387.500000	342.500000	308.000000	313.000000	311.500000
75%	1464.000000	1384.000000	1301.500000	1222.250000	1227.500000
max	9330.000000	5915.000000	5719.000000	5936.000000	5777.000000

	OCTOBER	NOVEMBER	DECEMBER	TOTAL
count	490.000000	490.000000	490.000000	490.000000
mean	928.491837	933.044898	967.153061	11434.502041
std	1274.226151	1269.391981	1337.868111	15948.689470
min	0.000000	0.000000	0.000000	0.000000
25%	30.250000	28.250000	30.000000	357.750000
50%	311.500000	331.000000	347.500000	4174.500000
75%	1287.000000	1246.500000	1365.250000	16463.500000
max	5516.000000	5367.000000	5562.000000	75480.000000

Check for outliers in file

```
In [34]: import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
data4 = pd.read_csv('only_road_accidents_data_month2.csv')

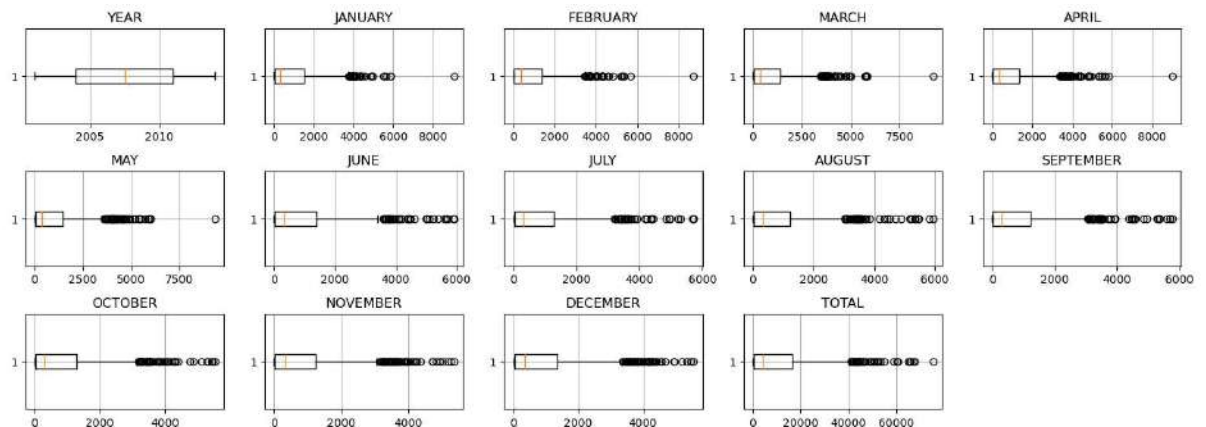
# Select columns with numeric data types (excluding non-numeric columns)
numeric_columns = data4.select_dtypes(include=['int64', 'float64'])

# Set the figure size
plt.figure(figsize=(15, 7))

# Create box plots for each numeric column
for i, column in enumerate(numeric_columns.columns):
    plt.subplot(4, 5, i + 1)
    plt.boxplot(data4[column], vert=False)
    plt.title(column)
    plt.grid(True)

# Adjust the layout
plt.tight_layout()

# Show the box plots
plt.show()
```



Data adequacy assessment summary

```
In [127]: if missing_values.sum() == 0 and duplicate_rows == 0:
print("\nData appears to be adequate.")
else:
print("\nData has issues that need to be addressed.")
```

Data appears to be adequate.

Models

linear Regression

```
In [4]: # Import necessary libraries
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Load your dataset (replace 'your_dataset.csv' with the actual dataset file)
data = pd.read_csv('only_road_accidents_data_month2.csv') # Replace with your

# Select the features and target variable
X = data[['JANUARY', 'FEBRUARY', 'MARCH', 'APRIL', 'MAY', 'JUNE', 'JULY', 'AUGU
y = data['TOTAL'] # Replace 'TOTAL' with your actual target variable

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create a Linear Regression model
model = LinearRegression()

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# Print the model coefficients and evaluation metrics
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
print("Mean Squared Error:", mse)
print("R-squared:", r2)
```

```
Coefficients: [1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
Intercept: 5.4569682106375694e-12
Mean Squared Error: 1.0079207449373342e-22
R-squared: 1.0
```

Random Forest

```
In [3]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load your dataset (replace 'your_dataset.csv' with the actual dataset file)
data = pd.read_csv('only_road_accidents_data_month2.csv') # Replace with your

# Select the features and target variable
X = data[['JANUARY', 'FEBRUARY', 'MARCH', 'APRIL', 'MAY', 'JUNE', 'JULY', 'AUGU
y = data['TOTAL'] # Replace 'TOTAL' with your actual target variable

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create a Random Forest classifier with 100 trees (you can adjust the number of
model = RandomForestClassifier(n_estimators=100, random_state=42)

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

# Print the model's accuracy and classification report
print("Accuracy:", accuracy)
print("Classification Report:\n", report)
```

Accuracy: 0.030612244897959183

Classification Report:

	precision	recall	f1-score	support
0	0.50	1.00	0.67	1
1	0.00	0.00	0.00	1
21	0.00	0.00	0.00	0
30	0.00	0.00	0.00	1
53	0.00	0.00	0.00	1
56	0.00	0.00	0.00	1
57	0.00	0.00	0.00	1
60	0.00	0.00	0.00	0
72	1.00	1.00	1.00	1
77	0.00	0.00	0.00	0
87	0.00	0.00	0.00	1
95	0.00	0.00	0.00	1
103	0.00	0.00	0.00	0
111	0.00	0.00	0.00	1
114	0.00	0.00	0.00	0
116	0.00	0.00	0.00	1
157	0.00	0.00	0.00	1
181	0.00	0.00	0.00	1
186	0.00	0.00	0.00	0
191	0.00	0.00	0.00	0
204	0.00	0.00	0.00	1
229	0.00	0.00	0.00	1
235	0.00	0.00	0.00	0
236	0.00	0.00	0.00	1
243	0.00	0.00	0.00	1
244	0.00	0.00	0.00	0
257	0.00	0.00	0.00	1
261	0.00	0.00	0.00	0
263	0.00	0.00	0.00	0
271	0.00	0.00	0.00	0
285	0.00	0.00	0.00	1
301	0.00	0.00	0.00	0
314	0.00	0.00	0.00	1
328	0.00	0.00	0.00	1
330	0.00	0.00	0.00	1
351	0.00	0.00	0.00	1
366	0.00	0.00	0.00	0
373	0.00	0.00	0.00	1
409	0.00	0.00	0.00	1
411	0.00	0.00	0.00	1
412	0.00	0.00	0.00	0
424	0.00	0.00	0.00	1
443	0.00	0.00	0.00	1
456	0.00	0.00	0.00	1
465	0.00	0.00	0.00	0
477	0.00	0.00	0.00	1
493	0.00	0.00	0.00	1
502	0.00	0.00	0.00	0
521	0.00	0.00	0.00	1
523	0.00	0.00	0.00	0
528	0.00	0.00	0.00	0
536	0.00	0.00	0.00	1
538	0.00	0.00	0.00	0

544	0.00	0.00	0.00	1
564	0.00	0.00	0.00	0
600	0.00	0.00	0.00	1
691	0.00	0.00	0.00	1
771	0.00	0.00	0.00	0
801	0.00	0.00	0.00	1
818	0.00	0.00	0.00	0
865	0.00	0.00	0.00	1
880	1.00	1.00	1.00	1
901	0.00	0.00	0.00	0
1003	0.00	0.00	0.00	1
1252	0.00	0.00	0.00	0
1286	0.00	0.00	0.00	1
1295	0.00	0.00	0.00	0
1409	0.00	0.00	0.00	1
1451	0.00	0.00	0.00	1
1455	0.00	0.00	0.00	1
1461	0.00	0.00	0.00	1
1480	0.00	0.00	0.00	0
1586	0.00	0.00	0.00	0
1698	0.00	0.00	0.00	0
1808	0.00	0.00	0.00	1
2002	0.00	0.00	0.00	0
2036	0.00	0.00	0.00	1
2899	0.00	0.00	0.00	1
3077	0.00	0.00	0.00	0
3078	0.00	0.00	0.00	0
3325	0.00	0.00	0.00	1
3749	0.00	0.00	0.00	0
3901	0.00	0.00	0.00	1
4170	0.00	0.00	0.00	1
4179	0.00	0.00	0.00	0
4230	0.00	0.00	0.00	1
4334	0.00	0.00	0.00	1
4536	0.00	0.00	0.00	0
4905	0.00	0.00	0.00	0
4987	0.00	0.00	0.00	1
5778	0.00	0.00	0.00	1
5874	0.00	0.00	0.00	0
5888	0.00	0.00	0.00	0
6012	0.00	0.00	0.00	1
6180	0.00	0.00	0.00	1
6323	0.00	0.00	0.00	1
6406	0.00	0.00	0.00	0
6424	0.00	0.00	0.00	0
6535	0.00	0.00	0.00	0
7023	0.00	0.00	0.00	1
7111	0.00	0.00	0.00	0
7144	0.00	0.00	0.00	1
7802	0.00	0.00	0.00	1
7837	0.00	0.00	0.00	1
7913	0.00	0.00	0.00	0
8869	0.00	0.00	0.00	0
9282	0.00	0.00	0.00	1
9398	0.00	0.00	0.00	1
9531	0.00	0.00	0.00	0
9580	0.00	0.00	0.00	0

9680	0.00	0.00	0.00	0
9699	0.00	0.00	0.00	1
10198	0.00	0.00	0.00	1
10262	0.00	0.00	0.00	1
10553	0.00	0.00	0.00	0
11043	0.00	0.00	0.00	1
11430	0.00	0.00	0.00	0
11547	0.00	0.00	0.00	1
12140	0.00	0.00	0.00	0
12152	0.00	0.00	0.00	0
12181	0.00	0.00	0.00	1
13085	0.00	0.00	0.00	1
13511	0.00	0.00	0.00	0
13954	0.00	0.00	0.00	0
13959	0.00	0.00	0.00	1
14468	0.00	0.00	0.00	0
14689	0.00	0.00	0.00	1
14725	0.00	0.00	0.00	1
18102	0.00	0.00	0.00	0
18740	0.00	0.00	0.00	1
19712	0.00	0.00	0.00	0
19808	0.00	0.00	0.00	1
20571	0.00	0.00	0.00	1
22969	0.00	0.00	0.00	1
23245	0.00	0.00	0.00	0
23348	0.00	0.00	0.00	1
23704	0.00	0.00	0.00	0
23885	0.00	0.00	0.00	1
24639	0.00	0.00	0.00	0
25038	0.00	0.00	0.00	1
28444	0.00	0.00	0.00	1
29173	0.00	0.00	0.00	0
30127	0.00	0.00	0.00	1
30199	0.00	0.00	0.00	0
32577	0.00	0.00	0.00	1
32860	0.00	0.00	0.00	0
37269	0.00	0.00	0.00	0
37289	0.00	0.00	0.00	1
40273	0.00	0.00	0.00	0
41323	0.00	0.00	0.00	1
43022	0.00	0.00	0.00	0
43280	0.00	0.00	0.00	1
46220	0.00	0.00	0.00	0
46252	0.00	0.00	0.00	1
47120	0.00	0.00	0.00	1
51025	0.00	0.00	0.00	1
51978	0.00	0.00	0.00	1
52508	0.00	0.00	0.00	0
55145	0.00	0.00	0.00	0
59117	0.00	0.00	0.00	1
60409	0.00	0.00	0.00	1
60794	0.00	0.00	0.00	1
64996	0.00	0.00	0.00	1
65873	0.00	0.00	0.00	1
66238	0.00	0.00	0.00	0
67757	0.00	0.00	0.00	1
75480	0.00	0.00	0.00	1

accuracy			0.03	98
macro avg	0.01	0.02	0.02	98
weighted avg	0.03	0.03	0.03	98

C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Recall and F-score are ill-defined and being set to 0.0 in labels with no true samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Recall and F-score are ill-defined and being set to 0.0 in labels with no true samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Recall and F-score are ill-defined and being set to 0.0 in labels with no true samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))

SVM Model

```
In [2]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# Load your dataset (replace 'your_dataset.csv' with the actual dataset file)
data = pd.read_csv('only_road_accidents_data_month2.csv') # Replace with your

# Select the features and target variable
X = data[['JANUARY', 'FEBRUARY', 'MARCH', 'APRIL', 'MAY', 'JUNE', 'JULY', 'AUGU
y = data['TOTAL'] # Replace 'TOTAL' with your actual target variable

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create an SVM classifier with a Linear kernel
model = SVC(kernel='linear', random_state=42) # You can choose the kernel type

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

# Print the model's accuracy and classification report
print("Accuracy:", accuracy)
print("Classification Report:\n", report)
```

Accuracy: 0.030612244897959183

Classification Report:

	precision	recall	f1-score	support
0	0.50	1.00	0.67	1
1	0.00	0.00	0.00	1
30	0.00	0.00	0.00	1
35	0.00	0.00	0.00	0
39	0.00	0.00	0.00	0
53	0.00	0.00	0.00	1
56	0.00	0.00	0.00	1
57	0.00	0.00	0.00	1
60	0.00	0.00	0.00	0
72	0.50	1.00	0.67	1
87	0.00	0.00	0.00	1
91	0.00	0.00	0.00	0
95	0.00	0.00	0.00	1
103	0.00	0.00	0.00	0
111	0.00	0.00	0.00	1
114	0.00	0.00	0.00	0
116	0.00	0.00	0.00	1
152	0.00	0.00	0.00	0
157	0.00	0.00	0.00	1
181	0.00	0.00	0.00	1
191	0.00	0.00	0.00	0
204	0.00	0.00	0.00	1
206	0.00	0.00	0.00	0
218	0.00	0.00	0.00	0
229	0.00	0.00	0.00	1
236	0.00	0.00	0.00	1
243	0.00	0.00	0.00	1
244	0.00	0.00	0.00	0
257	0.00	0.00	0.00	1
261	0.00	0.00	0.00	0
285	0.00	0.00	0.00	1
301	0.00	0.00	0.00	0
314	0.00	0.00	0.00	1
328	0.00	0.00	0.00	1
330	0.00	0.00	0.00	1
351	0.00	0.00	0.00	1
366	0.00	0.00	0.00	0
373	0.00	0.00	0.00	1
409	0.00	0.00	0.00	1
411	0.00	0.00	0.00	1
412	0.00	0.00	0.00	0
424	0.00	0.00	0.00	1
443	0.00	0.00	0.00	1
456	0.00	0.00	0.00	1
465	0.00	0.00	0.00	0
477	0.00	0.00	0.00	1
493	0.00	0.00	0.00	1
502	0.00	0.00	0.00	0
521	0.00	0.00	0.00	1
523	0.00	0.00	0.00	0
536	0.00	0.00	0.00	1
538	0.00	0.00	0.00	0
544	0.00	0.00	0.00	1

600	0.00	0.00	0.00	1
691	0.00	0.00	0.00	1
771	0.00	0.00	0.00	0
801	0.00	0.00	0.00	1
818	0.00	0.00	0.00	0
865	0.00	0.00	0.00	1
880	1.00	1.00	1.00	1
995	0.00	0.00	0.00	0
1003	0.00	0.00	0.00	1
1252	0.00	0.00	0.00	0
1286	0.00	0.00	0.00	1
1383	0.00	0.00	0.00	0
1409	0.00	0.00	0.00	1
1451	0.00	0.00	0.00	1
1455	0.00	0.00	0.00	1
1461	0.00	0.00	0.00	1
1484	0.00	0.00	0.00	0
1780	0.00	0.00	0.00	0
1808	0.00	0.00	0.00	1
2036	0.00	0.00	0.00	1
2145	0.00	0.00	0.00	0
2899	0.00	0.00	0.00	1
3077	0.00	0.00	0.00	0
3099	0.00	0.00	0.00	0
3325	0.00	0.00	0.00	1
3901	0.00	0.00	0.00	1
3995	0.00	0.00	0.00	0
4170	0.00	0.00	0.00	1
4179	0.00	0.00	0.00	0
4230	0.00	0.00	0.00	1
4334	0.00	0.00	0.00	1
4536	0.00	0.00	0.00	0
4905	0.00	0.00	0.00	0
4987	0.00	0.00	0.00	1
5778	0.00	0.00	0.00	1
5874	0.00	0.00	0.00	0
5888	0.00	0.00	0.00	0
6012	0.00	0.00	0.00	1
6180	0.00	0.00	0.00	1
6323	0.00	0.00	0.00	1
6391	0.00	0.00	0.00	0
6406	0.00	0.00	0.00	0
7023	0.00	0.00	0.00	1
7111	0.00	0.00	0.00	0
7144	0.00	0.00	0.00	1
7211	0.00	0.00	0.00	0
7802	0.00	0.00	0.00	1
7837	0.00	0.00	0.00	1
7913	0.00	0.00	0.00	0
9110	0.00	0.00	0.00	0
9195	0.00	0.00	0.00	0
9282	0.00	0.00	0.00	1
9398	0.00	0.00	0.00	1
9531	0.00	0.00	0.00	0
9580	0.00	0.00	0.00	0
9699	0.00	0.00	0.00	1
10198	0.00	0.00	0.00	1

10262	0.00	0.00	0.00	1
10316	0.00	0.00	0.00	0
11043	0.00	0.00	0.00	1
11161	0.00	0.00	0.00	0
11547	0.00	0.00	0.00	1
12152	0.00	0.00	0.00	0
12181	0.00	0.00	0.00	1
13085	0.00	0.00	0.00	1
13511	0.00	0.00	0.00	0
13959	0.00	0.00	0.00	1
14457	0.00	0.00	0.00	0
14468	0.00	0.00	0.00	0
14689	0.00	0.00	0.00	1
14725	0.00	0.00	0.00	1
18102	0.00	0.00	0.00	0
18740	0.00	0.00	0.00	1
19712	0.00	0.00	0.00	0
19808	0.00	0.00	0.00	1
19999	0.00	0.00	0.00	0
20571	0.00	0.00	0.00	1
22969	0.00	0.00	0.00	1
23245	0.00	0.00	0.00	0
23348	0.00	0.00	0.00	1
23704	0.00	0.00	0.00	0
23885	0.00	0.00	0.00	1
24639	0.00	0.00	0.00	0
25038	0.00	0.00	0.00	1
28444	0.00	0.00	0.00	1
29173	0.00	0.00	0.00	0
30127	0.00	0.00	0.00	1
30199	0.00	0.00	0.00	0
32577	0.00	0.00	0.00	1
32860	0.00	0.00	0.00	0
37269	0.00	0.00	0.00	0
37289	0.00	0.00	0.00	1
40273	0.00	0.00	0.00	0
41323	0.00	0.00	0.00	1
42295	0.00	0.00	0.00	0
43280	0.00	0.00	0.00	1
46220	0.00	0.00	0.00	0
46252	0.00	0.00	0.00	1
47120	0.00	0.00	0.00	1
51025	0.00	0.00	0.00	1
51978	0.00	0.00	0.00	1
52508	0.00	0.00	0.00	0
55145	0.00	0.00	0.00	0
59117	0.00	0.00	0.00	1
60409	0.00	0.00	0.00	1
60794	0.00	0.00	0.00	1
64996	0.00	0.00	0.00	1
65873	0.00	0.00	0.00	1
66238	0.00	0.00	0.00	0
67757	0.00	0.00	0.00	1
75480	0.00	0.00	0.00	1
accuracy			0.03	98
macro avg	0.01	0.02	0.01	98

weighted avg 0.02 0.03 0.02 98

```
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and bei
ng set to 0.0 in labels with no predicted samples. Use `zero_division` parame
ter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Recall and F-score are ill-defined and being
set to 0.0 in labels with no true samples. Use `zero_division` parameter to c
ontrol this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and bei
ng set to 0.0 in labels with no predicted samples. Use `zero_division` parame
ter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Recall and F-score are ill-defined and being
set to 0.0 in labels with no true samples. Use `zero_division` parameter to c
ontrol this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and bei
ng set to 0.0 in labels with no predicted samples. Use `zero_division` parame
ter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1344: UndefinedMetricWarning: Recall and F-score are ill-defined and being
set to 0.0 in labels with no true samples. Use `zero_division` parameter to c
ontrol this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
```

Decision Trees

```
In [1]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load your dataset (replace 'your_dataset.csv' with the actual dataset file)
data = pd.read_csv('only_road_accidents_data_month2.csv') # Replace with your

# Select the features and target variable
X = data[['JANUARY', 'FEBRUARY', 'MARCH', 'APRIL', 'MAY', 'JUNE', 'JULY', 'AUGU
y = data['TOTAL'] # Replace 'TOTAL' with your actual target variable

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Create a Decision Tree classifier
model = DecisionTreeClassifier(random_state=42)

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

# Print the model's accuracy and classification report
print("Accuracy:", accuracy)
print("Classification Report:\n", report)
```

Accuracy: 0.02040816326530612

Classification Report:

	precision	recall	f1-score	support
0	0.50	1.00	0.67	1
1	0.00	0.00	0.00	1
30	0.00	0.00	0.00	1
38	0.00	0.00	0.00	0
41	0.00	0.00	0.00	0
42	0.00	0.00	0.00	0
53	0.00	0.00	0.00	1
56	0.00	0.00	0.00	1
57	0.00	0.00	0.00	1
60	0.00	0.00	0.00	0
72	0.00	0.00	0.00	1
77	0.00	0.00	0.00	0
87	0.00	0.00	0.00	1
95	0.00	0.00	0.00	1
103	0.00	0.00	0.00	0
111	0.00	0.00	0.00	1
114	0.00	0.00	0.00	0
116	0.00	0.00	0.00	1
125	0.00	0.00	0.00	0
157	0.00	0.00	0.00	1
180	0.00	0.00	0.00	0
181	0.00	0.00	0.00	1
186	0.00	0.00	0.00	0
189	0.00	0.00	0.00	0
191	0.00	0.00	0.00	0
204	0.00	0.00	0.00	1
229	0.00	0.00	0.00	1
235	0.00	0.00	0.00	0
236	0.00	0.00	0.00	1
243	0.00	0.00	0.00	1
257	0.00	0.00	0.00	1
261	0.00	0.00	0.00	0
271	0.00	0.00	0.00	0
285	0.00	0.00	0.00	1
301	0.00	0.00	0.00	0
314	0.00	0.00	0.00	1
328	0.00	0.00	0.00	1
330	0.00	0.00	0.00	1
351	0.00	0.00	0.00	1
366	0.00	0.00	0.00	0
373	0.00	0.00	0.00	1
409	0.00	0.00	0.00	1
411	0.00	0.00	0.00	1
424	0.00	0.00	0.00	1
441	0.00	0.00	0.00	0
443	0.00	0.00	0.00	1
456	0.00	0.00	0.00	1
468	0.00	0.00	0.00	0
477	0.00	0.00	0.00	1
493	0.00	0.00	0.00	1
502	0.00	0.00	0.00	0
520	0.00	0.00	0.00	0
521	0.00	0.00	0.00	1

523	0.00	0.00	0.00	0
536	0.00	0.00	0.00	1
538	0.00	0.00	0.00	0
544	0.00	0.00	0.00	1
600	0.00	0.00	0.00	1
602	0.00	0.00	0.00	0
691	0.00	0.00	0.00	1
735	0.00	0.00	0.00	0
801	1.00	1.00	1.00	1
834	0.00	0.00	0.00	0
865	0.00	0.00	0.00	1
880	0.00	0.00	0.00	1
888	0.00	0.00	0.00	0
995	0.00	0.00	0.00	0
1003	0.00	0.00	0.00	1
1286	0.00	0.00	0.00	1
1375	0.00	0.00	0.00	0
1409	0.00	0.00	0.00	1
1451	0.00	0.00	0.00	1
1455	0.00	0.00	0.00	1
1461	0.00	0.00	0.00	1
1479	0.00	0.00	0.00	0
1510	0.00	0.00	0.00	0
1808	0.00	0.00	0.00	1
1982	0.00	0.00	0.00	0
2036	0.00	0.00	0.00	1
2281	0.00	0.00	0.00	0
2899	0.00	0.00	0.00	1
3077	0.00	0.00	0.00	0
3259	0.00	0.00	0.00	0
3325	0.00	0.00	0.00	1
3901	0.00	0.00	0.00	1
4170	0.00	0.00	0.00	1
4179	0.00	0.00	0.00	0
4228	0.00	0.00	0.00	0
4230	0.00	0.00	0.00	1
4288	0.00	0.00	0.00	0
4334	0.00	0.00	0.00	1
4987	0.00	0.00	0.00	1
5011	0.00	0.00	0.00	0
5778	0.00	0.00	0.00	1
5888	0.00	0.00	0.00	0
6012	0.00	0.00	0.00	1
6180	0.00	0.00	0.00	1
6323	0.00	0.00	0.00	1
6406	0.00	0.00	0.00	0
6424	0.00	0.00	0.00	0
6455	0.00	0.00	0.00	0
7023	0.00	0.00	0.00	1
7111	0.00	0.00	0.00	0
7144	0.00	0.00	0.00	1
7802	0.00	0.00	0.00	1
7837	0.00	0.00	0.00	1
8184	0.00	0.00	0.00	0
8441	0.00	0.00	0.00	0
8566	0.00	0.00	0.00	0
8769	0.00	0.00	0.00	0

9110	0.00	0.00	0.00	0
9282	0.00	0.00	0.00	1
9398	0.00	0.00	0.00	1
9699	0.00	0.00	0.00	1
10198	0.00	0.00	0.00	1
10262	0.00	0.00	0.00	1
10320	0.00	0.00	0.00	0
10553	0.00	0.00	0.00	0
11043	0.00	0.00	0.00	1
11354	0.00	0.00	0.00	0
11547	0.00	0.00	0.00	1
12152	0.00	0.00	0.00	0
12181	0.00	0.00	0.00	1
13085	0.00	0.00	0.00	1
13657	0.00	0.00	0.00	0
13959	0.00	0.00	0.00	1
14457	0.00	0.00	0.00	0
14468	0.00	0.00	0.00	0
14689	0.00	0.00	0.00	1
14725	0.00	0.00	0.00	1
17105	0.00	0.00	0.00	0
18031	0.00	0.00	0.00	0
18740	0.00	0.00	0.00	1
19712	0.00	0.00	0.00	0
19808	0.00	0.00	0.00	1
20571	0.00	0.00	0.00	1
22969	0.00	0.00	0.00	1
23245	0.00	0.00	0.00	0
23348	0.00	0.00	0.00	1
23591	0.00	0.00	0.00	0
23885	0.00	0.00	0.00	1
24639	0.00	0.00	0.00	0
25038	0.00	0.00	0.00	1
28444	0.00	0.00	0.00	1
30127	0.00	0.00	0.00	1
31947	0.00	0.00	0.00	0
32577	0.00	0.00	0.00	1
32860	0.00	0.00	0.00	0
35661	0.00	0.00	0.00	0
36439	0.00	0.00	0.00	0
37289	0.00	0.00	0.00	1
41066	0.00	0.00	0.00	0
41323	0.00	0.00	0.00	1
43280	0.00	0.00	0.00	1
44382	0.00	0.00	0.00	0
46252	0.00	0.00	0.00	1
46586	0.00	0.00	0.00	0
47120	0.00	0.00	0.00	1
48887	0.00	0.00	0.00	0
51025	0.00	0.00	0.00	1
51975	0.00	0.00	0.00	0
51978	0.00	0.00	0.00	1
59117	0.00	0.00	0.00	1
60409	0.00	0.00	0.00	1
60794	0.00	0.00	0.00	1
64996	0.00	0.00	0.00	1
65873	0.00	0.00	0.00	1

66238	0.00	0.00	0.00	0
67250	0.00	0.00	0.00	0
67757	0.00	0.00	0.00	1
75480	0.00	0.00	0.00	1
accuracy			0.02	98
macro avg	0.01	0.01	0.01	98
weighted avg	0.02	0.02	0.02	98

C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Recall and F-score are ill-defined and being set to 0.0 in labels with no true samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Recall and F-score are ill-defined and being set to 0.0 in labels with no true samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))
C:\Users\lenovo\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Recall and F-score are ill-defined and being set to 0.0 in labels with no true samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))